جزوهٔ آموزشی · کنکور دکتری ۱۴۰۶ · مجموعهٔ ۲۳۵۴

درس ۴ از ۱۰ سرفصل

مبانی کامپیوتر و برنامه‌سازی

درس‌نامهٔ کوتاه و فصل‌به‌فصل، مثال و تلهٔ تستی، تمرین با پاسخ، و آزمون آخر هر فصل از سؤال‌های واقعی کنکور

```
{ i++; }
```

۶	۲۳	۶۰
فصل	درس	تست تألیفیِ اجراشده با gcc

فصل‌های این جزوه

۱ نوع داده، عملگرها، اولویت و تبدیل نوع (زبان C)

۲ شرط، switch، حلقه، break و continue

۳ تابع، ارسال پارامتر، حوزهٔ متغیر، static و ماکرو

۴ بازگشت و ردگیری فراخوانی

۵ اشاره‌گر، آرایه و رشته

۶ struct، union، حافظهٔ پویا و لیست پیوندی

راهنمای استفاده از جزوه

هر فصل با یک برگهٔ رنگی شروع می‌شود و همیشه همین ترتیب را دارد: درس‌ها ← آزمون فصل ← کارت خلاصه ← پاسخ‌نامه.

✏️ مثال
یک مسئلهٔ حل‌شده برای اینکه قاعده را در عمل ببینی.

⚠️ تلهٔ تستی
جایی که طراح کنکور بیشتر از همه داوطلب‌ها را غافلگیر می‌کند.

💡 نکته و قاعده
میان‌بُرها و قاعده‌هایی که سر جلسه وقت می‌خرند.

✍️ تمرین
آخر هر درس؛ پاسخ‌ها در پاسخ‌نامهٔ آخر همان فصل است (با شمارهٔ صفحه).

📝 آزمون فصل
سؤال‌ها در کارت‌های جدا، با منبع و مربعی برای علامت غلط یا حدسی.

🗂️ کارت خلاصه
چکیدهٔ یک‌صفحه‌ای فصل برای مرور شب قبل از آزمون.

چرخهٔ مطالعهٔ هر فصل (یک بلوک ۹۰ دقیقه‌ای)

۱	۲	۳	۴	۵
پیش‌آزمون	**درس‌نامه**	**تمرین**	**آزمون فصل**	**دفتر خطا**
۵ دقیقه	**۳۵ دقیقه**	**۳۰ دقیقه**	**+۱۵ دقیقه**	**۵ دقیقه**
سؤال اول آزمون فصل را قبل از خواندن امتحان کن	مثال‌ها را قبل از دیدن حل، خودت امتحان کن	بدون نگاه به پاسخ‌نامه حل کن	با زمان، مثل جلسهٔ کنکور	هر غلط یا حدسی = یک سطر در دفتر خطا

قانون طلایی: خواندن یعنی بلد بودن نیست. جزوه‌ای که روان خوانده شد ولی آزمونش را نزدی، یاد گرفته نشده است. معیار فقط **درصد تست** است، با فرمول نمرهٔ منفی: **(۳ × درست − غلط) ÷ (۳ × کل)**.

منبع سؤال‌ها: تست‌های آزمون فصل‌های این درس **تألیفی**اند (این درس در کنکور ارشد کامپیوتر سؤال سابقه ندارد) و خروجی همهٔ کدها با کامپایلر gcc اجرا و تأیید شده است. فهرست قابل کلیک است: روی هر عنوان بزن تا به همان صفحه بروی. در نمایشگر PDF، نشانک‌ها (bookmark) هم فصل‌ها و درس‌ها را نشان می‌دهند.

فهرست

روی هر سطر بزن تا مستقیم به همان صفحه بروی.

۱ **نوع داده، عملگرها، اولویت و تبدیل نوع (زبان C)** ۱۰ تست تألیفی ۵

درس ۱ برنامهٔ C از صفر و خواندن خروجی ۵
درس ۲ انواع داده ۸
درس ۳ عملگرهای حسابی و افزایشی ۱۰
درس ۴ اولویت، شرکت‌پذیری و اتصال کوتاه ۱۲
درس ۵ تبدیل نوع ۱۴
📝 آزمون فصل ۱۶ | 🗂️ کارت خلاصه ۱۸ | 🔑 پاسخ‌نامه ۱۹

۲ **شرط، switch، حلقه، break و continue** ۱۰ تست تألیفی ۲۱

درس ۱ if و else ۲۱
درس ۲ switch ۲۳
درس ۳ حلقه‌ها ۲۵
درس ۴ break و continue ۲۷
📝 آزمون فصل ۳۰ | 🗂️ کارت خلاصه ۳۲ | 🔑 پاسخ‌نامه ۳۳

۳ **تابع، ارسال پارامتر، حوزهٔ متغیر، static و ماکرو** ۱۰ تست تألیفی ۳۵

درس ۱ تعریف و فراخوانی تابع ۳۵
درس ۲ ارسال پارامتر ۳۶
درس ۳ حوزه (scope) و طول عمر ۳۷
درس ۴ ماکرو (#define) ۳۹
📝 آزمون فصل ۴۱ | 🗂️ کارت خلاصه ۴۳ | 🔑 پاسخ‌نامه ۴۴

۴ **بازگشت و ردگیری فراخوانی** ۱۰ تست تألیفی ۴۵

درس ۱ ساختار تابع بازگشتی و ترتیب چاپ ۴۵
درس ۲ شمردن فراخوانی‌ها و توابع کلاسیک ۴۷
📝 آزمون فصل ۵۰ | 🗂️ کارت خلاصه ۵۲ | 🔑 پاسخ‌نامه ۵۳

۵ **اشاره‌گر، آرایه و رشته** ۱۰ تست تألیفی ۵۴

درس ۱ آرایه از صفر ۵۴
درس ۲ اشاره‌گر ۵۵
درس ۳ آرایه و حساب اشاره‌گر ۵۶
درس ۴ رشته ۵۷
📝 آزمون فصل ۵۹ | 🗂️ کارت خلاصه ۶۱ | 🔑 پاسخ‌نامه ۶۲

۶ struct، union، حافظهٔ پویا و لیست پیوندی ۱۰ تست تألیفی ۶۳

درس ۱ struct ۶۳
درس ۲ اندازهٔ struct و union ۶۴
درس ۳ حافظهٔ پویا ۶۵
درس ۴ لیست پیوندی در C ۶۷

📝 آزمون فصل ۶۹ | 🗂️ کارت خلاصه ۷۱ | 🔑 پاسخ‌نامه ۷۲

فصل ۱

۱ نوع داده، عملگرها، اولویت و تبدیل نوع (زبان C)

۵ درس | ۱۰ تست تألیفی در آزمون فصل

💡 **چرا این فصل؟** درس «مبانی کامپیوتر و برنامه‌سازی» در سرفصل دکتری ۱۴۰۶ (مجموعهٔ ۲۳۵۴) آمده، ولی در کنکور ارشد کامپیوتر درسی با این نام نبوده و سؤال سابقه ندارد. تیپ رایج این درس **«خروجی این کد چیست؟»** است و تقریباً همهٔ تله‌ها از همین فصل می‌آیند: تقسیم صحیح، اولویت عملگرها، اتصال کوتاه، و تبدیل نوع.

📐 **فرض‌ها** C استاندارد، `int` ۴ بایتی، `char` ۱ بایتی (حالت رایج در تست‌ها).

✅ **اعتبار** خروجی همهٔ کدهای این فصل با کامپایلر `gcc` اجرا و چک شده است. (مثال‌های تکمیلی: `کد/مثال-ها.c`)

⏱️ **زمان مطالعه** یک بلوک ۹۰ دقیقه‌ای.

نقشهٔ فصل: ۱ برنامهٔ C از صفر و خواندن خروجی | ۲ انواع داده | ۳ عملگرهای حسابی و افزایشی | ۴ اولویت، شرکت‌پذیری و اتصال کوتاه | ۵ تبدیل نوع | 📝 آزمون فصل

درس ۱ برنامهٔ C از صفر و خواندن خروجی

این درس برای چیست؟

تقریباً همهٔ تست‌های این درس یک شکل دارند: **«خروجی این کد چیست؟»**. برای جواب دادن لازم نیست برنامه‌نویس حرفه‌ای باشی. کافی است مثل کامپیوتر، خط‌به‌خط جلو بروی و مقدار متغیرها را روی کاغذ نگه داری. این درس همین مهارت را از صفر می‌سازد؛ بقیهٔ کتاب روی آن بنا شده است.

برنامه از بالا به پایین اجرا می‌شود

```c
#include <stdio.h>          // کتابخانهٔ ورودی و خروجی (برای printf)
int main(void) {            // اجرای برنامه همیشه از main شروع می‌شود
    int x = 5;              // خط ۱
    x = x + 2;              // خط ۲
    printf("%d", x);        // خط ۳: چاپ می‌کند 7
    return 0;
}
```

کامپیوتر خط ۱، بعد خط ۲، بعد خط ۳ را اجرا می‌کند. هیچ خطی «از قبل» یا «هم‌زمان» اجرا نمی‌شود (مگر حلقه و شرط، که در فصل ۲ می‌آیند). در تست‌ها معمولاً فقط بدنهٔ main را می‌بینی.

متغیر یعنی یک جعبهٔ اسم‌دار در حافظه

`int x = 5;` یعنی: «یک جعبه به اسم x بساز که در آن عدد صحیح (int) جا می‌شود، و ۵ را در آن بگذار.»

- جعبه در هر لحظه **فقط یک مقدار** دارد. مقدار تازه، مقدار قبلی را پاک می‌کند.
- نوع جعبه (int، char، double ...) تعیین می‌کند چه چیزی در آن جا می‌شود (درس ۲).

علامت = یعنی «بریز در»، نه «مساوی است»

`x = x + 2` در ریاضی بی‌معنی است، ولی در C یعنی: **اول سمت راست را با مقدار فعلی حساب کن، بعد نتیجه را در جعبهٔ سمت چپ بریز.** با `x = 5`، سمت راست ۷ می‌شود و ۷ در x می‌نشیند.

> 💡 **قاعده**
>
> در هر خط انتساب، اول **همهٔ** سمت راست با مقدارهای **قبل از** این خط حساب می‌شود، بعد سمت چپ عوض می‌شود.

چاپ با printf: الگوهای درصد

`printf` متن داخل گیومه را عیناً چاپ می‌کند و هر الگوی `%...` را با مقدار بعدی جایگزین می‌کند، به ترتیب.

الگو	چه چیزی چاپ می‌کند	مثال	خروجی
`%d`	عدد صحیح	`printf("%d", 7)`	`7`
`%c`	یک کاراکتر	`printf("%c", 65)`	`A`
`%f`	عدد اعشاری با ۶ رقم اعشار	`printf("%f", 2.5)`	`2.500000`
`%.2f`	اعشاری با ۲ رقم اعشار	`printf("%.2f", 2.5)`	`2.50`
`%s`	رشته	`printf("%s", "hi")`	`hi`
`%u`	عدد صحیح بی‌علامت	`printf("%u", 7u)`	`7`
`%zu`	خروجی sizeof	`printf("%zu", sizeof(int))`	`4`
`\n`	رفتن به خط بعد		

> ✏️ **مثال**
>
> `printf("%d%d", 1, 2);` چاپ می‌کند `12`، نه `1 2`. فاصله فقط وقتی چاپ می‌شود که خودت در گیومه گذاشته باشی: `printf("%d %d", 1, 2)` ⇐ `1 2`.

مهارت اصلی: جدول ردگیری

برای هر کد، جدولی بکش که هر ستونش یک متغیر است و هر سطرش یک خط کد. بعد از هر خط، مقدار جدید را بنویس.

✏️ مثال (جابه‌جایی بدون متغیر کمکی)

```c
int a = 3, b = 4;
a = a + b;
b = a - b;
a = a - b;
printf("%d %d", a, b);
```

خط	a	b	توضیح
`int a = 3, b = 4;`	۳	۴	
`a = a + b;`	۷	۴	`3 + 4`
`b = a - b;`	۷	۳	`7 - 4` ؛ از a **جدید** استفاده شد
`a = a - b;`	۴	۳	`7 - 3`

خروجی: `4 3` . یعنی مقدارها جابه‌جا شدند.

⚠️ تلهٔ جابه‌جایی

`x = y; y = x;` جابه‌جا **نمی‌کند**. با `x = 2, y = 5` : خط اول x را ۵ می‌کند و ۲ برای همیشه پاک می‌شود؛ خط دوم ۵ را دوباره در y می‌ریزد. خروجی `5 5` . برای جابه‌جایی درست یک جعبهٔ کمکی لازم است: `t = x; x = y; y = t;`.

✍️ تمرین درس ۱

اول خودت حل کن، بعد پاسخ‌نامه

```c
// 1
int p = 10, q = 3;
p = p - q;  q = p + q;
printf("%d %d", p, q);
// 2
printf("%d-%d", 4, 5);
printf("%c%c", 72, 105);
// 3
int k = 1;
k = k * 2;  k = k * 2;  k = k + k;
printf("%d", k);
```

🔑 پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: **صفحهٔ ۱۹**

درس ۲ انواع داده

از صفر: بیت و بایت

حافظه از خانه‌های خیلی کوچکی به اسم **بیت** ساخته شده که هر کدام فقط ۰ یا ۱ است. هشت بیت را یک **بایت** می‌گویند.

- با n بیت دقیقاً 2^n الگوی مختلف می‌سازی. یک بایت (۸ بیت) ⇐ $2^8 = 256$ مقدار.
- پس هر نوع داده، چون تعداد بایت محدودی دارد، **بازهٔ محدودی** از عددها را جا می‌دهد.

✏️ **مثال**

`int` چهار بایت یعنی ۳۲ بیت دارد ⇐ $2^{32} \approx 4.3$ میلیارد الگو. نصف برای منفی‌ها، نصف برای صفر و مثبت‌ها ⇐ حدود منفی ۲٫۱ میلیارد تا مثبت ۲٫۱ میلیارد.

نوع	اندازهٔ رایج	نکته
`char`	۱ بایت	در واقع یک عدد صحیح کوچک است
`short`	۲ بایت	
`int`	۴ بایت	
`long`	۴ یا ۸ بایت	به سیستم بستگی دارد (لینوکس ۶۴ بیتی: ۸)
`double` / `float`	۸ / ۴ بایت	`0.1` در C به‌طور پیش‌فرض `double` است

✏️ **مثال (خروجی روی لینوکس ۶۴ بیتی)**

```c
printf("%zu %zu %zu %zu", sizeof(char), sizeof(short), sizeof(int), sizeof(double));
// 1 2 4 8
printf("%zu %zu", sizeof(0.1), sizeof(0.1f));
// 8 4
```

`sizeof` اندازهٔ هر نوع را بر حسب بایت می‌دهد. `0.1` به‌طور پیش‌فرض double است (۸ بایت)؛ با پسوند `f` می‌شود float (۴ بایت).

char در واقع عدد است

کامپیوتر فقط عدد می‌فهمد. هر کاراکتر با یک عدد ذخیره می‌شود (جدول **ASCII**). لازم نیست کل جدول را حفظ کنی؛ همین سه بازه کافی است:

کاراکترها	کدها
`'0'` تا `'9'`	۴۸ تا ۵۷
`'A'` تا `'Z'`	۶۵ تا ۹۰

کاراکترها	کدها
`'a'` تا `'z'`	۹۷ تا ۱۲۲

نتیجه‌ها: `'a' - 'A' = 32` (فاصلهٔ حروف بزرگ و کوچک)، `'7' - '0' = 7` (تبدیل رقم کاراکتری به عدد)، و `'z' - 'a' = 25`.

⚠️ **تلهٔ تستی**

`'7'` (کاراکتر، کد ۵۵) با `7` (عدد) فرق دارد. `'7' + 1` برابر ۵۶ است، یعنی کاراکتر `'8'`.

✏️ **مثال (char عدد است)**

```
char c = 'B';
printf("%c %d %c", c + 1, c - 'A', 'G' + 32);   // C 1 g
```

`'B' + 1 = 67` همان `'C'` است؛ `'B' - 'A' = 1` شمارهٔ حرف در الفباست؛ و جمع با ۳۲، حرف بزرگ را کوچک می‌کند. یک عدد را با `%c` چاپ کنی کاراکترش می‌آید و با `%d` خود عدد.

unsigned: کیلومترشمار ماشین

نوع `unsigned` فقط صفر و مثبت را نگه می‌دارد و وقتی از سقفش رد شود **دور می‌زند**، مثل کیلومترشمار ماشینی که از `999999` به `000000` برمی‌گردد. یعنی حساب به پیمانهٔ 2^n انجام می‌شود.

✏️ **مثال**

`unsigned char` (۸ بیت، بازهٔ ۰ تا ۲۵۵) با مقدار ۲۵۵، بعد از `++` صفر می‌شود. و `unsigned int` صفر، بعد از `-1` بزرگ‌ترین عدد خودش می‌شود: $2^{32} - 1$.

⚠️ **تلهٔ تستی**

سرریز نوع **باعلامت** (مثل int معمولی) «رفتار تعریف‌نشده» است و تست درست آن را نمی‌پرسد.

sizeof عبارت داخلش را اجرا نمی‌کند

`sizeof` فقط به **نوع** عبارت نگاه می‌کند و آن را اجرا نمی‌کند. پس بعد از `sizeof(x++)` مقدار x عوض **نمی‌شود**. همچنین در C، `sizeof('A')` برابر ۴ است (چون `'A'` از نوع int است)؛ در C++ برابر ۱ است.

✍️ تمرین درس ۲ (خروجی چیست؟) — اول خودت حل کن، بعد پاسخ‌نامه

```c
// 1
printf("%d %d %d", 'A', 'a' - 'A', '7' - '0');
// 2
unsigned char c = 250;  c = c + 10;  printf("%d", c);
// 3
int x = 7;  size_t s = sizeof(x++);  printf("%zu %d", s, x);
// 4
unsigned int u = 0;  u = u - 1;  printf("%u", u);
```

🔑 پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: **صفحهٔ ۱۹**

درس ۳ عملگرهای حسابی و افزایشی

تقسیم صحیح: مثل تقسیم آب‌نبات

وقتی هر دو عدد صحیح‌اند، `/` تقسیم **صحیح** است: فقط سهم کامل هر نفر را می‌دهد و `%` باقی‌ماندهٔ همان تقسیم را.

✏️ مثال

۱۷ آب‌نبات بین ۵ نفر: هر نفر ۳ تا (`17/5 = 3`) و ۲ تا می‌ماند (`17%5 = 2`). چک: `3·5 + 2 = 17`.

- تقسیم صحیح **به سمت صفر بریده می‌شود**: `7/2 = 3` و `-7/2 = -3` (نه -۴).
- **علامت `%` همان علامت عدد اول است:** `-7 % 2 = -1` و `7 % -2 = 1`.
- همیشه برقرار است: `a == (a/b)*b + a%b`.

✏️ مثال

- با `n = 1234`: `n % 10 = 4` (رقم یکان) و `n / 10 = 123` (حذف رقم یکان). حلقه‌های «جمع ارقام» همین دو کار را تکرار می‌کنند.
- `-7 = (-3)·2 + (-1)` ✓ `(-7/2)*2 + (-7%2)`.

++ و --: دو کار در یک علامت

`i++` دو کار انجام می‌دهد: (۱) یک **مقدار** به عبارت می‌دهد، و (۲) i را یکی زیاد می‌کند. فرق `i++` و `++i` فقط در این است که کدام مقدار به عبارت داده شود:

شکل	مقداری که در عبارت استفاده می‌شود	بعد از آن
`i++`	**مقدار قدیمی** i	i یکی زیاد می‌شود
`++i`	**مقدار جدید** (بعد از زیاد شدن)	i زیاد شده است

✏️ مثال

`int i = 5; printf("%d ", i++); printf("%d", i);` ⇐ `5 6` . اولی مقدار قدیمی را چاپ کرد؛ تا خط دوم i شش شده بود.

✏️ مثال (هر دو شکل)

`int i = 3; int a = i++; int b = ++i;` ⇐ `a = 3` (مقدار قدیمی)، بعد i چهار می‌شود و `i++` آن را ۵ می‌کند: `b = 5` و `i = 5` .

✏️ مثال

`int i = 5; int j = i++ + 2;` ⇒ `j = 7` و `i = 6` .

⚠️ رفتار تعریف‌نشده

`i = i++;` یا `printf("%d %d", i++, i++);` . تغییر یک متغیر دو بار در یک عبارت، خروجی قطعی ندارد. اگر در تستی دیدی، گزینهٔ «وابسته به کامپایلر / تعریف‌نشده» را در نظر بگیر.

انتساب ترکیبی

`a += 3` یعنی `a = a + 3` . نکته این است که **کل سمت راست اول حساب می‌شود**: در `a *= 3 + 4` یعنی `a = a *` `(3+4)`. پس اگر a برابر ۲ باشد، نتیجه ۱۴ است نه ۱۰.

✏️ مثال

`int x = 2; x -= 1 + 1;` ⇐ `x = 2 - (1 + 1) = 0` ، نه `(2 - 1) + 1 = 2` .

✍️ تمرین درس ۳ — اول خودت حل کن، بعد پاسخ‌نامه

```c
// 1
printf("%d %d %d %d", -7/2, -7%2, 7%-2, 7/-2);
// 2
int i = 5;  int j = i++ + 2;  int k = ++i * 2;  printf("%d %d %d", i, j, k);
// 3
int a = 17, b = 5;  printf("%d %d", a/b*b + a%b, a % b * 2);
```

🔑 پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: **صفحهٔ ۱۹**

درس ۴ اولویت، شرکت‌پذیری و اتصال کوتاه

از صفر: همان قانون ریاضی دبیرستان

در ریاضی یاد گرفتی `2 + 3 × 4 = 14` است، نه ۲۰، چون ضرب **اولویت** دارد. C هم همین کار را می‌کند، فقط عملگرهایش بیشترند. وقتی عبارتی پرانتز ندارد، جدول اولویت می‌گوید کامپیوتر پرانتزها را کجا می‌گذارد.

> ✏️ **مثال**
>
> `2 + 3 * 4` ⇐ ۱۴؛ `(2 + 3) * 4` ⇐ ۲۰؛ `2 * 3 % 4` ⇐ `*` و `%` هم‌اولویت‌اند و از چپ حساب می‌شوند: `6 % 4 = 2`.

شرکت‌پذیری (جهت): وقتی دو عملگر هم‌اولویت کنار هم‌اند، جهت تعیین می‌کند کدام اول اجرا شود. بیشترشان از چپ به راست‌اند: `10 - 4 - 3 = (10 - 4) - 3 = 3`. ولی انتساب از راست به چپ است: در `a = b = 5` اول `b = 5` و بعد `a = 5`.

جدول اولویت (از بالا به پایین؛ فقط آنچه در تست می‌آید)

اولویت	عملگرها	جهت
۱	`()` `[]` `→` `.` و `++` `--` پسوندی	چپ به راست
۲	یکانی: `!` `~` `++` `--` `-` `*` `&` `sizeof` `(type)`	**راست به چپ**
۳	`*` `/` `%`	چپ به راست
۴	`+` `-`	
۵	`<<` `>>`	
۶	`<` `≤` `>` `≥`	
۷	`==` `≠`	
۸	`&`	
۹	`^`	
۱۰	`|`	
۱۱	`&&`	
۱۲	`||`	
۱۳	`?:`	**راست به چپ**
۱۴	`=` `+=` `-=` …	**راست به چپ**
۱۵	`,`	چپ به راست

> 💡 **جملهٔ حفظی**
>
> «یکانی، ضرب، جمع، شیفت، مقایسه، تساوی، بیتی (& ^ |)، منطقی (&& ||)، شرط، انتساب، کاما.»

یادآوری درست و غلط در C: نتیجهٔ هر مقایسه یک **عدد** است: ۱ اگر درست و ۰ اگر نادرست. هر عدد غیرصفر «درست» حساب می‌شود.

> 💡 **روش**
>
> پرانتزهای نامرئی را خودت بگذار، از بالاترین اولویت شروع کن، و بعد از داخل به بیرون حساب کن.

> ✏️ **مثال (ارزیابی قدم‌به‌قدم با جدول)**
>
> `int r = 2 + 3 * 4 > 10 && !0;`
>
> ۱. یکانی: `!0` = 1 .
>
> ۲. ضرب: `3 * 4` = 12 .
>
> ۳. جمع: `2 + 12` = 14 .
>
> ۴. مقایسه: `14 > 10` ⇐ ۱.
>
> ۵. منطقی: `1 && 1` ⇐ ۱. پس `r` = 1 .

پنج تلهٔ معروف

۱. `a < b < c` **بازه را چک نمی‌کند.** اول `a<b` حساب می‌شود که ۰ یا ۱ است، بعد آن عدد با c مقایسه می‌شود. مثال: `5 > 4 > 3` برابر `(1) > 3` یعنی ۰ است.

۲. **بیتی از تساوی ضعیف‌تر است:** `x & 3 == 3` یعنی `x & (3==3)` یعنی `x & 1` .

۳. **شیفت از جمع ضعیف‌تر است:** `1 << 2 + 1` یعنی `1 << 3` یعنی ۸.

۴. **اتصال کوتاه:** در `A && B` اگر A نادرست باشد B **اجرا نمی‌شود**؛ در `A || B` اگر A درست باشد B اجرا نمی‌شود. پس `++`داخل B ممکن است اصلاً اجرا نشود.

۵. `=` **به جای** `==` `if (x = 0)` x را صفر می‌کند و شرط همیشه نادرست است. عملگر کاما هم مقدار **آخرین** عبارت را برمی‌گرداند: `(1, 2, 3)` برابر ۳ است.

اتصال کوتاه به زبان ساده

وقتی جواب از نیمهٔ اول معلوم شد، کامپیوتر نیمهٔ دوم را **اصلاً نگاه نمی‌کند**. مثل اینکه بخواهی «اگر در باز است **و** چراغ روشن است وارد شو»: اگر در قفل باشد، دیگر سراغ کلید چراغ نمی‌روی.

- `0 &&هرچیز` ⇐ ۰، بدون اجرای «هرچیز».
- `1 ||هرچیز` ⇐ ۱، بدون اجرای «هرچیز».

✏️ مثال (استفادهٔ درست از اتصال کوتاه)

`if (x ≠ 0 && 10 / x > 1)` . اگر x صفر باشد، شرط اول نادرست است و تقسیم بر صفر اصلاً اجرا نمی‌شود.

✍️ تمرین درس ۴ — اول خودت حل کن، بعد پاسخ‌نامه

```c
// 1
int a = 3, b = 4, c = 5;  printf("%d %d", a < b < c, c > b > a);
// 2
int x = 6 & 3 == 3;  printf("%d", x);
// 3
int i = 0, j = 0;
if (i++ && j++) ;   printf("%d %d ", i, j);
if (i++ || j++) ;   printf("%d %d", i, j);
// 4
printf("%d %d", !5 == 0, 1 << 2 + 1);
// 5
int y = 5;  y = y > 3 ? y << 1 : y >> 1;  printf("%d", y);
```

🔑 پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: **صفحهٔ ۱۹**

درس ۵ تبدیل نوع

از صفر: نوع جواب را عملوندها تعیین می‌کنند

کامپیوتر برای هر عمل، اول به **نوع دو عدد دو طرفش** نگاه می‌کند و بعد تصمیم می‌گیرد تقسیم صحیح کند یا اعشاری. به متغیری که قرار است جواب در آن ریخته شود **نگاه نمی‌کند**. این بزرگ‌ترین تلهٔ این درس است.

✏️ مثال

`7/2` ⇐ ۳ (هر دو صحیح)؛ `7/2.0` ⇐ `3.5` (یکی اعشاری)؛ `(double)7/2` ⇐ `3.5` (اول ۷ اعشاری شد)؛ `(double)(7/2)` ⇐ `3.0` (اول تقسیم صحیح انجام شد و ۳ شد، بعد اعشاری).

قواعد

۱. **نوع عملیات از روی عملوندها تعیین می‌شود، نه از روی متغیری که نتیجه در آن ذخیره می‌شود.** در `double d = 5/2;` تقسیم صحیح است و `d = 2.0` .

۲. اگر یکی از عملوندها اعشاری باشد، عملیات اعشاری می‌شود: `(double)5/2 = 2.5` ، ولی `(double)(5/2) = 2.0` .

۳. **تبدیل اعشاری به صحیح به سمت صفر می‌رُود:** `(int)2.7 = 2` و `(int)-2.7 = -2` . گرد نمی‌کند.

۴. **مقایسهٔ signed با unsigned:** عدد signed به unsigned تبدیل می‌شود. پس `-1 < 1u` **نادرست** است، چون -۱ به `4294967295` تبدیل می‌شود (کیلومترشمار درس ۲).

۵. **`float` با `double`:** `float f = 0.1;` و بعد `f == 0.1` نادرست است، چون `0.1` در float و double با دقت متفاوت ذخیره می‌شود.

✏️ مثال (میانگین)

```
int s = 7, n = 2;
double avg  = (double)s / n;    // 3.5
double avg2 = s / n;            // 3.0  (تقسیم صحیح، بعد تبدیل)
int k = 3.99;                   // 3    (بریدن، نه گرد کردن)
```

✍️ تمرین درس ۵

اول خودت حل کن، بعد پاسخ‌نامه

```
// 1
double d = 5/2;  printf("%f %.1f %.1f", d, (double)5/2, (double)(5/2));
// 2
printf("%s", (-1 < 1u) ? "yes" : "no");
// 3
int x = 'a' + 1.5;  printf("%d", x);
// 4
printf("%.1f %.1f", 3/2*2.0, 3*2.0/2);
// 5
float f = 0.1;  printf("%s", f == 0.1 ? "equal" : "not equal");
```

🔑 پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: **صفحهٔ ۱۹**

خلاصهٔ روش حل تست‌های «خروجی چیست؟»

۱. جدول ردگیری بکش: یک ستون برای هر متغیر.

۲. هر خط را با مقدارهای **فعلی** حساب کن؛ اول سمت راست، بعد انتساب.

۳. هر جا پرانتز نبود، با جدول اولویت پرانتز بگذار.

۴. سه چیز را همیشه چک کن: تقسیم صحیح یا اعشاری؟ `i++` یا `++i`؟ اتصال کوتاه جلوی اجرای چیزی را گرفته؟

۵. خروجی را دقیقاً مثل printf بنویس: فاصله‌ها، خط‌ها، و الگوهای `%` .

📝 آزمون فصل ۱

تست‌های تألیفی (خروجی همه با gcc اجرا و تأیید شده)

۱۰ سؤال | ⏱️ زمان: ۱۵ دقیقه | 🔑 کلید و راه‌حل: **صفحهٔ ۲۰**

این درس در کنکور ارشد کامپیوتر سؤال سابقه ندارد. پس این ۱۰ تست **تألیفی** هستند و در قالب تست کنکور نوشته شده‌اند. خروجی همهٔ کدها با `gcc` اجرا و تأیید شده است.

خروجی هر کد چیست؟

✔️ مثل جلسهٔ کنکور: زمان بگیر، روی هیچ سؤالی بیشتر از ۴ دقیقه نمان، و در مربع کنار سؤال‌های غلط یا حدسی علامت بزن تا در دور دوم فقط همان‌ها را حل کنی.

۱ تألیفی — غلط یا حدسی؟

```c
int a = 10, b = 3;
printf("%d %d", a / b, -a % b);
```

۱) `3 1`　۲) `3 -1`　۳) `4 -1`　۴) `3 2`

۲ تألیفی — غلط یا حدسی؟

```c
int x = 5, y = 0;
if (x > 3 || ++y) x++;
printf("%d %d", x, y);
```

۱) `6 1`　۲) `6 0`　۳) `5 1`　۴) `5 0`

۳ تألیفی — غلط یا حدسی؟

```c
int a = 1, b = 2, c = 3;
printf("%d", a < b < c == c > b > a);
```

۱) `0`　۲) `1`　۳) `3`　۴) خطای کامپایل

۴ تألیفی — غلط یا حدسی؟

```c
unsigned int u = 1;
int i = -1;
printf("%s", i < u ? "less" : "not less");
```

۱) `less`　۲) `not less`　۳) خطای کامپایل　۴) رفتار تعریف‌نشده

۵ تألیفی

غلط یا حدسی؟

```
double d = 7 / 2 + 7 / 2.0;
printf("%.1f", d);
```

۱) 7.0
۲) 6.5
۳) 6.0
۴) 7.5

۶ تألیفی

غلط یا حدسی؟

```
int x = 0x0F;
printf("%d", x & 0x3C | 1 << 4);
```

۱) 28
۲) 12
۳) 208
۴) 13

۷ تألیفی

غلط یا حدسی؟

(فرض: `int` ۴ بایتی)

```
int i = 3;
int j = sizeof(i++) + i;
printf("%d", j);
```

۱) 7
۲) 8
۳) 4
۴) خطای کامپایل

۸ تألیفی

غلط یا حدسی؟

```
char c = 'B';
c = c + ('a' - 'A');
printf("%c %d", c, c - 'a');
```

۱) b 1
۲) B 1
۳) b 33
۴) b 0

۹ تألیفی

غلط یا حدسی؟

```
int a = 5, b;
b = (a++, a * 2);
printf("%d %d", a, b);
```

۱) 6 12
۲) 5 10
۳) 6 10
۴) 5 12

۱۰ تألیفی

غلط یا حدسی؟

```c
int k = 0;
int r = (k = 3) && (k = 0) || (k = 5);
printf("%d %d", r, k);
```

۱) 1 5
۲) 0 0
۳) 1 3
۴) 0 5

🗂️ کارت خلاصهٔ فصل ۱

برای مرور سریع، شب قبل از آزمون

- ✔ `char` عدد است: A=۶۵، a=۹۷، ۰=۴۸. `unsigned` دور می‌زند. `sizeof` اجرا نمی‌کند.
- ✔ `/` صحیح به سمت صفر می‌بُرد. علامت `%` همان علامت عدد اول است.
- ✔ `i++` اول مقدار قدیمی را می‌دهد. دو بار تغییر یک متغیر در یک عبارت رفتار تعریف‌نشده است.
- ✔ اولویت: یکانی > `* / %` > `+ -` > شیفت > مقایسه > تساوی > `&` > `^` > `|` > `&&` > `||` > `?:` > `=` > `,`
- ✔ `a<b<c` بازه نیست. `&&` و `||` اتصال کوتاه دارند.
- ✔ نوع عملیات را عملوندها تعیین می‌کنند. signed در مقایسه با unsigned تبدیل می‌شود. float را با `==` مقایسه نکن.

پاسخ‌نامهٔ فصل ۱

تا تمرین‌ها و آزمون فصل را خودت حل نکرده‌ای، این صفحه‌ها را نخوان.

تمرین درس ۱

برگشت ← صفحهٔ ۷

۱. `7 10` . بعد از خط اول `p = 7` ؛ بعد `q = 7 + 3 = 10` (از p جدید استفاده شد).

۲. `4-5Hi` . خط دوم بدون فاصله ادامه می‌دهد؛ ۷۲ کد حرف `H` و ۱۰۵ کد حرف `i` است.

۳. `8` . مقدار k به ترتیب ۱، ۲، ۴ و بعد `4 + 4 = 8` است.

تمرین درس ۲ (خروجی چیست؟)

برگشت ← صفحهٔ ۱۰

۱. `65 32 7`

۲. `4` (۲۶۰ به پیمانهٔ ۲۵۶)

۳. `4 7` (x اجرا نشد)

۴. `4294967295` (یعنی `2³² - 1`)

تمرین درس ۳

برگشت ← صفحهٔ ۱۱

۱. `-3 -1 1 -3`

۲. `7 7 14` (بعد از خط دوم i=۶ و j=۷؛ بعد `i ++`آن را ۷ می‌کند و `k = 14`)

۳. `17 4` (`*` و `%` هم‌اولویت‌اند و از چپ به راست حساب می‌شوند: `4 = 2*(17%5)`)

تمرین درس ۴

برگشت ← صفحهٔ ۱۴

۱. `1 0`

۲. `0` (`6 & 1 = 0`)

۳. `1 0 2 0` . در شرط اول، `i++` مقدار ۰ را برمی‌گرداند، پس `j++` اجرا نمی‌شود. در شرط دوم، `i++` مقدار ۱ را برمی‌گرداند، پس باز `j++` اجرا نمی‌شود.

۴. `1 8` (`0 = 5!` و `0 == 0` یعنی ۱)

۵. `10`

تمرین درس ۵

برگشت ← صفحهٔ ۱۵

۱. `2.000000 2.5 2.0`

۲. `no`

۳. `98` (`97 + 1.5 = 98.5` که بریده می‌شود)

۴. `2.0 3.0` . در اولی `3/2` اول حساب می‌شود و ۱ است. در دومی `3*2.0 = 6.0` و بعد تقسیم بر ۲.

۵. `not equal`

آزمون فصل ۱ — کلید و راه‌حل

برگشت ← صفحهٔ ۱۶

سؤال ۱	سؤال ۲	سؤال ۳	سؤال ۴	سؤال ۵	سؤال ۶	سؤال ۷	سؤال ۸	سؤال ۹	سؤال ۱۰
۲	۲	۱	۲	۲	۱	۱	۱	۱	۱

۱. `10/3 = 3` . منفی یکانی اولویت بالاتری دارد، پس `-a % b` یعنی `(-10) % 3` که برابر `-1` است (علامت عدد اول).

۲. `x > 3` درست است، پس `y ++` به خاطر اتصال کوتاه اجرا نمی‌شود. خروجی `6 0` .

۳. مقایسه‌ها از `==` قوی‌ترند. سمت چپ: `(1<2)=1` و `1<3` می‌شود ۱. سمت راست: `(3>2)=1` و `1>1` می‌شود ۰. پس `1 == 0` یعنی **۰**.

۴. -۱ به unsigned تبدیل می‌شود و `4294967295` می‌شود، که از ۱ بزرگ‌تر است. خروجی `not less` . این رفتار کاملاً تعریف‌شده است.

۵. `7/2 = 3` (صحیح) و `7/2.0 = 3.5` . جمع **۶.۵**.

۶. ترتیب اولویت: شیفت، بعد `&` ، بعد `|` . `1<<4 = 16` و `0x0F & 0x3C = 0x0C = 12` . پس `12 | 16 = 28` .

۷. `sizeof` اجرا نمی‌کند، پس i همان ۳ می‌ماند. جواب `4 + 3 = 7` .

۸. `'B' + 32 = 'b'` و `'b' - 'a' = 1` .

۹. کاما نقطهٔ ترتیب است: اول `a++` اجرا می‌شود و a می‌شود ۶، بعد `b = 6*2 = 12` .

۱۰. `&&` از `||` قوی‌تر است. `(k=3)` مقدار ۳ (درست) دارد، `(k=0)` مقدار ۰ (نادرست)، پس `&&` می‌شود ۰. بعد `(k=5)` اجرا می‌شود که درست است، پس `r = 1` و `k = 5` . چون `&&` و `||` نقطهٔ ترتیب دارند، رفتار تعریف‌شده است.

فصل ۲

۲ شرط، switch، حلقه، break و continue

۴ درس | ۱۰ تست تألیفی در آزمون فصل

💡 **چرا این فصل؟** تست‌های «خروجی کد» تقریباً همیشه یک حلقه یا switch دارند. تله‌ها محدود و تکراری‌اند: else آویزان، سقوط در switch، بعد از for یا if، و شمردن تکرارها.

✅ **اعتبار** خروجی همهٔ کدها با `gcc` اجرا و چک شده است. (مثال‌های تکمیلی: `کد/مثال-ها.c`)

⏱️ **زمان مطالعه** یک بلوک ۹۰ دقیقه‌ای.

نقشهٔ فصل: ۱ if و else | ۲ switch | ۳ حلقه‌ها | ۴ break و continue | 📝 آزمون فصل

درس ۱ if و else

از صفر: دوراهی

تا اینجا برنامه خط‌به‌خط پایین می‌رفت. `if` یک **دوراهی** می‌سازد: «اگر شرط درست است این کار را بکن، وگرنه (else) آن کار را.»

```
if (شرط)
    دستور_۱;        //فقط اگر شرط درست باشد
else
    دستور_۲;        //فقط اگر شرط نادرست باشد
دستور_۳;            //در هر دو حالت
```

یادآوری از فصل ۱: شرط در C یک **عدد** است. صفر یعنی نادرست و **هر عدد غیرصفر** (حتی منفی) یعنی درست.

آکولاد `{ }` مرز بدنه را تعیین می‌کند

بدون آکولاد، فقط **یک** دستورِ بعد از if جزو آن است، هر قدر هم که تورفتگی چیز دیگری نشان بدهد. کامپایلر تورفتگی را نمی‌بیند.

✏️ **مثال**

```
int x = 0;
if (x > 0)
    printf("A");
    printf("B");    // نیست! این خط جزو if
```

خروجی `B` . شرط نادرست است و فقط `printf("A")` رد می‌شود؛ خط بعدی همیشه اجرا می‌شود.

زنجیرهٔ else if: اولین درست برنده است

در زنجیرهٔ `if … else if … else` ، شرط‌ها از بالا یکی‌یکی چک می‌شوند و **فقط اولین** شاخه‌ای که درست باشد اجرا می‌شود. بقیه، حتی اگر درست باشند، دیده نمی‌شوند.

✏️ **مثال**

```
int n = 75;
if (n ⩾ 90)      printf("A");
else if (n ⩾ 70) printf("B");    // اولین شرط درست
else             printf("C");
```

خروجی `B` . شرط `n ⩾ 70` درست بود و زنجیره همان‌جا تمام شد.

سه قاعدهٔ تستی

۱. **هر عدد غیرصفر «درست» است**، حتی عدد منفی. فقط صفر «نادرست» است.

۲. **else آویزان:** else به **نزدیک‌ترین if بدون else** می‌چسبد، نه به if ای که تورفتگی نشان می‌دهد.

۳. **بعد از if** `;` یعنی بدنهٔ if خالی است. دستور بعدی **همیشه** اجرا می‌شود.

✏️ **مثال (قاعدهٔ ۱ و ۳)**

```
if (-3) printf("A");        // چاپ می‌شود: -3 غیرصفر است
if (0)  printf("B");        // چاپ نمی‌شود
if (5 > 9); printf("C");    // همان ; خالی است چاپ می‌شود: بدنهٔ if
```

خروجی: `AC` .

✏️ مثال (else آویزان)

```c
int a = 0, b = 1;
if (a)
    if (b) printf("X");
else printf("Y");      // این else مال if (b) است!
printf("Z");
```

خروجی فقط `Z` است. با پرانتزهای واقعی کد این است: `if (a) { if (b) X; else Y; }` . چون a صفر است، کل بلوک داخلی اجرا نمی‌شود.

💡 روش

وقتی کد تورفتگی گمراه‌کننده دارد، خودت آکولادها را بر اساس قاعده‌های بالا بگذار و بعد ردگیری کن.

✍️ تمرین درس ۱ — اول خودت حل کن، بعد پاسخ‌نامه

```c
// 1
int x = 3;  if (x > 5);  printf("A");
// 2
int y = 0;  if (y = 5) printf("%d", y);
// 3
int z = -2;  printf("%s", z ? "true" : "false");
```

🔑 پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: **صفحهٔ ۳۳**

درس ۲ switch

از صفر: آسانسوری که ترمز ندارد

`switch` مقدار یک متغیر را با برچسب‌های `case` مقایسه می‌کند. با آسانسور تصورش کن: آسانسور در طبقه‌ای که با مقدار جور است **باز می‌شود** و تو وارد می‌شوی، ولی بعد از آن **همهٔ طبقه‌های پایین‌تر** را هم طی می‌کنی، تا وقتی به یک `break` برسی که تو را بیرون می‌اندازد. اسم این رفتار «سقوط» (fall-through) است.

قواعد

- مقدار switch و برچسب‌های case باید **صحیح یا کاراکتر** و **ثابت** باشند. بازه (مثل `case 1..5`) در C استاندارد نیست.
- **سقوط:** اگر case با `break` تمام نشود، اجرا به case بعدی **ادامه می‌یابد** و دیگر شرط آن چک نمی‌شود.
- `default` یعنی «هیچ case ای جور نشد». می‌تواند هر جایی باشد؛ وقتی اجرا شود، بعد از آن هم سقوط ادامه دارد.

✏️ مثال (قدم‌به‌قدم)

```
int k = 2, s = 0;
switch (k) {
    case 1: s += 1;
    case 2: s += 2;          // از اینجا وارد می‌شویم
    case 3: s += 3; break;   // سقوط تا اینجا
    case 4: s += 4;
    default: s += 10;
}
```

۱. k برابر ۲ است ⇐ ورود از `case 2` ؛ `case 1` اصلاً اجرا نمی‌شود.

۲. `s += 2` ⇐ ۲. break نیست ⇐ سقوط به `case 3` (بدون چک کردن ۳).

۳. `s += 3` ⇐ ۵. break ⇐ خروج.

جواب `s = 5` . با `k = 4` می‌شد `4 + 10 = 14` (سقوط به default).

✏️ مثال (استفادهٔ مفید از سقوط)

```
switch (ch) {
    case 'a': case 'e': case 'i': case 'o': case 'u':
        printf("vowel"); break;
    default:
        printf("consonant");
}
```

با `ch = 'e'` از `case 'e'` وارد می‌شود، تا `printf` سقوط می‌کند و `vowel` چاپ می‌شود. چند case پشت سر هم مثل «یا» عمل می‌کنند.

💡 روش

(۱) جای ورود را پیدا کن (case جور، یا default). (۲) از همان‌جا خط‌به‌خط پایین برو و همهٔ دستورها را اجرا کن. (۳) با اولین break یا آخر switch بایست.

✍️ تمرین درس ۲

اول خودت حل کن، بعد پاسخ‌نامه

```c
int k = 7, s = 0;
switch (k) {
    default: s += 10;
    case 1:  s += 1; break;
    case 2:  s += 2;
}
printf("%d", s);
```

🔑 پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: **صفحهٔ ۳۳**

درس ۳ حلقه‌ها

از صفر: تکرار یک کار

حلقه یعنی «این کار را تا وقتی شرط درست است تکرار کن». سه شکل دارد که در واقع یک کار می‌کنند و فقط **زمان چک کردن شرط** فرقشان است.

آناتومی حلقهٔ for

`for (شروع; شرط; گام) بدنه;` دقیقاً به این ترتیب اجرا می‌شود:

۱. **شروع** فقط **یک بار** اجرا می‌شود.

۲. **شرط** چک می‌شود. اگر نادرست بود ⇐ حلقه تمام.

۳. **بدنه** اجرا می‌شود.

۴. **گام** اجرا می‌شود و برمی‌گردیم به قدم ۲.

✏️ **مثال (جدول ردگیری)**

```
int s = 0, i;
for (i = 1; i ⩽ 4; i++) s += i;
printf("%d %d", s, i);
```

دور	i (موقع چک شرط)	شرط	s بعد از بدنه
۱	۱	درست	۱
۲	۲	درست	۳
۳	۳	درست	۶
۴	۴	درست	۱۰
—	۵	**نادرست**	حلقه تمام

خروجی `10 5` . ⚠️ بعد از حلقه، i برابر **۵** است، نه ۴: همان مقداری که شرط را نقض کرد.

سه شکل حلقه

حلقه	نکته
`for (init; cond; step)`	بعد از هر تکرار `step` اجرا می‌شود و بعد `cond` چک می‌شود
`while (cond)`	شرط **قبل از** بدنه چک می‌شود ⇐ ممکن است **صفر بار** اجرا شود
`do { … } while (cond);`	شرط **بعد از** بدنه چک می‌شود ⇐ **حداقل یک بار** اجرا می‌شود

✏️ **مثال برای هر سطر جدول**

```
for (int k = 0; k < 3; k++) printf("%d", k);    // 012
int j = 5;  while (j < 3) printf("X");           // هیچ (صفر بار)
int m = 5;  do printf("Y"); while (m < 3);       // Y (یک بار)
```

✏️ **مثال (while با ضرب)**

`int k = 1; while (k < 20) k *= 3;` ⇐ k به ترتیب ۱، ۳، ۹، ۲۷. در ۲۷ شرط نادرست می‌شود ⇐ بعد از حلقه `k = 27` .

تله‌ها

۱. **بعد از for:** `for (i = 0; i < 5; i++);` ; حلقه‌ای با بدنهٔ خالی است. بعد از آن `i == 5` است و دستور بعدی **یک بار** اجرا می‌شود.

۲. **شمردن تکرار:** تعداد تکرارهای `for (i = a; i ⩽ b; i += d)` برابر `⌊(b-a)/d⌋ + 1` است.

۳. **پسوندی در شرط:** در `while (i++ < 3)` مقایسه با مقدار **قدیمی** i انجام می‌شود، ولی بدنه مقدار **جدید** را می‌بیند.

۴. **کاما در for:** `for (i = 0, j = 10; i < j; i += 2, j -= 3)` دو متغیر را هم‌زمان جلو می‌برد.

۵. **unsigned:** شرط `u ⩾ 0` برای unsigned همیشه درست است و حلقه بی‌نهایت می‌شود.

✏️ **مثال برای تله‌ها**

- **شمردن:** `for (i = 2; i ⩽ 20; i += 5)` با `i = 2, 7, 12, 17` ⇐ `⌊18/5⌋ + 1 = 4` تکرار؛ بعد از حلقه `i = 22`.
- **کاما:** `for (i = 0, j = 10; i < j; i += 2, j -= 3)` جفت‌های `(0, 10)` و `(2, 7)` را می‌بینند. بعد `(4, 4)` شرط را نقض می‌کند ⇐ ۲ تکرار.
- **unsigned:** `for (unsigned u = 2; u ⩾ 0; u--)` مقادیر `2, 1, 0, 4294967295, 4294967294, …` را می‌گیرد و هیچ‌وقت تمام نمی‌شود.

💡 **روش برای حلقه‌های تودرتو**

برای هر مقدار متغیر بیرونی، حلقهٔ درونی را **کامل** اجرا کن. اگر تعداد دورها خواسته شده، برای هر دور بیرونی تعداد دور درونی را بنویس و جمع بزن (همان روش «شمردن حلقه‌ها» در ساختمان داده).

✍️ **تمرین درس ۳** — اول خودت حل کن، بعد پاسخ‌نامه

```
// 1
int i = 10;  do { printf("%d ", i); i++; } while (i < 5);
// 2
int c = 0;  for (i = 1; i ⩽ 100; i += 3) c++;   printf("%d %d", c, i);
// 3
i = 0;  while (i++ < 3) printf("%d ", i);   printf("| %d", i);
// 4
i = 5;  while (i ⟶ 0) printf("%d", i);        // یعنی (i--) > 0
// 5
int c2 = 0;  for (int a = 0; a < 4; a++) for (int b = a; b < 4; b++) c2++;
```

🔑 پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: **صفحهٔ ۳۳**

درس ۴ — break و continue

از صفر: «برو بیرون» و «برو دور بعد»

- `break` یعنی **همین حالا از اتاق بیرون برو**: حلقه (یا switch) تمام می‌شود و اجرا از خط بعد از آن ادامه پیدا می‌کند.

- `continue` یعنی **بقیهٔ این دور را ول کن و برو سراغ دور بعد**: حلقه تمام نمی‌شود.

دستور	کار
`break`	فقط از **درونی‌ترین** حلقه یا switch بیرون می‌رود
`continue`	بقیهٔ بدنه را رد می‌کند. در `for` بعدش **step** اجرا می‌شود؛ در `while` مستقیم سراغ شرط می‌رود

✏️ مثال (هر دو با هم)

```
int s = 0;
for (int i = 1; i ⩽ 10; i++) {
    if (i % 2 == 0) continue;    // زوج‌ها رد می‌شوند
    if (i > 7) break;            // با 9 خارج می‌شویم
    s += i;
}
```

ردگیری: s=۱ ⇐ i=۱؛ i=۲ رد؛ s=۴ ⇐ i=۳؛ i=۴ رد؛ s=۹ ⇐ i=۵؛ i=۶ رد؛ s=۱۶ ⇐ i=۷؛ i=۸ رد؛ break ⇐ i=۹. جواب `s = 1 + 3 + 5 + 7 = 16`.

⚠️ تلهٔ تستی

در `while` اگر `i++` بعد از `continue` باشد، ممکن است اجرا نشود و حلقه بی‌نهایت شود. در `for` این مشکل نیست، چون step در خود for است.

✏️ مثال (continue در while)

```
int i = 0;
while (i < 5) {
    if (i == 2) continue;    // همیشه ۲ می‌ماند: حلقهٔ بی‌نهایت
    i++;
}
```

همین منطق با `for (i = 0; i < 5; i++) { if (i == 2) continue; }` مشکلی ندارد، چون `i++` بخش step است و بعد از continue هم اجرا می‌شود.

⚠️ break فقط یک لایه

در دو حلقهٔ تودرتو، break داخل حلقهٔ درونی فقط درونی را تمام می‌کند؛ بیرونی دور بعدش را ادامه می‌دهد. break داخل switch هم فقط از switch بیرون می‌رود، نه از حلقه‌ای که switch در آن است.

✍️ تمرین درس ۴

اول خودت حل کن، بعد پاسخ‌نامه

```
// 1
int c = 0;
for (int i = 0; i < 3; i++)
    for (int j = 0; j < 3; j++) { if (j == 1) break; c++; }
// 2
int s = 0, i = 0;
while (i < 10) { i++; if (i % 3 == 0) continue; s += i; }
```

🔑 پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: **صفحهٔ ۳۳**

📝 آزمون فصل ۲

تست‌های تألیفی (خروجی همه با gcc اجرا و تأیید شده)

۱۰ سؤال | ⏱️ زمان: ۱۵ دقیقه | 🔑 کلید و راه‌حل: صفحهٔ ۳۳

این درس در کنکور ارشد کامپیوتر سؤال سابقه ندارد. پس این ۱۰ تست **تألیفی** هستند و خروجی همه با `gcc` تأیید شده است.

خروجی هر کد چیست؟

✔️ مثل جلسهٔ کنکور: زمان بگیر، روی هیچ سؤالی بیشتر از ۴ دقیقه نمان، و در مربع کنار سؤال‌های غلط یا حدسی علامت بزن تا در دور دوم فقط همان‌ها را حل کنی.

۱ تألیفی — غلط یا حدسی؟ ☐

```c
int x = 5, y = 10;
if (x > y)
    if (y > 0) x = 1;
else x = 2;
printf("%d", x);
```

۱) 5 ۲) 1 ۳) 2 ۴) خطای کامپایل

۲ تألیفی — غلط یا حدسی؟ ☐

```c
int n = 3, r = 0;
switch (n) {
    case 1: r++;
    case 3: r += 3;
    case 5: r += 5;
    default: r += 100;
}
printf("%d", r);
```

۱) 3 ۲) 8 ۳) 108 ۴) 103

۳ تألیفی — غلط یا حدسی؟ ☐

```c
int i = 0, s = 0;
do { s += i; i += 2; } while (i < 0);
printf("%d %d", s, i);
```

۱) `0 0` ۲) `0 2` ۳) `2 2` ۴) حلقه اجرا نمی‌شود و `0 0`

۴ تألیفی

غلط یا حدسی؟

```
int c = 0;
for (int i = 1; i ⩽ 20; i *= 3) c++;
printf("%d", c);
```

۱) 2
۲) 3
۳) 4
۴) 7

۵ تألیفی

غلط یا حدسی؟

```
int s = 0;
for (int i = 0; i < 10; i++) {
    if (i == 2) continue;
    if (i == 6) break;
    s += i;
}
printf("%d", s);
```

۱) 15
۲) 13
۳) 19
۴) 43

۶ تألیفی

غلط یا حدسی؟

```
int i = 10;
while (i-- > 7);
printf("%d", i);
```

۱) 7
۲) 6
۳) 9
۴) حلقهٔ بی‌نهایت

۷ تألیفی

غلط یا حدسی؟

```
int c = 0;
for (int i = 0; i < 5; i++)
    for (int j = 0; j < i; j++)
        if ((i + j) % 2) c++;
printf("%d", c);
```

۱) 4
۲) 5
۳) 6
۴) 10

۸ تألیفی

غلط یا حدسی؟

```
int x = -4, r = 0;
switch (x % 3) {
    case 1:  r = 1; break;
    case 2:  r = 2; break;
    case -1: r = 3; break;
    default: r = 4;
}
printf("%d", r);
```

۱) 1
۲) 2
۳) 3
۴) 4

۹ تألیفی

غلط یا حدسی؟

```
int i, j, c = 0;
for (i = 0, j = 10; i < j; i += 2, j -= 2) c++;
printf("%d %d %d", c, i, j);
```

۱) 3 6 4
۲) 2 4 6
۳) 3 4 6
۴) 5 10 0

۱۰ تألیفی

غلط یا حدسی؟

```
int n = 0;
for (int i = 1; i ⩽ 3; i++)
    for (int j = 1; j ⩽ 3; j++) {
        if (j == i) continue;
        if (j > i) break;
        n += 10 * i + j;
    }
printf("%d", n);
```

۱) 84
۲) 63
۳) 21
۴) 117

🗂️ کارت خلاصهٔ فصل ۲

برای مرور سریع، شب قبل از آزمون

- ✔ غیرصفر یعنی درست. else به نزدیک‌ترین if بدون else می‌چسبد. `;` بعد از if یا for یعنی بدنهٔ خالی.
- ✔ switch بدون break سقوط می‌کند. default هر جا باشد، بعدش هم سقوط ادامه دارد. علامت `%` مهم است.
- ✔ do-while حداقل یک بار اجرا می‌شود. تعداد تکرار `⌊(b-a)/d⌋ + 1` . `x++ < n` با مقدار قدیمی مقایسه می‌کند.
- ✔ break فقط درونی‌ترین حلقه را می‌شکند. continue در for به step می‌رود و در while به شرط.

پاسخ‌نامهٔ فصل ۲

تا تمرین‌ها و آزمون فصل را خودت حل نکرده‌ای، این صفحه‌ها را نخوان.

تمرین درس ۱

برگشت ← صفحهٔ ۲۳

۱. `A` (بدنهٔ if خالی است)

۲. `5` (انتساب، مقدار ۵ دارد که درست است)

۳. `true` (منفی هم غیرصفر است)

تمرین درس ۲

برگشت ← صفحهٔ ۲۵

`11` . هیچ case ای جور نیست، پس default اجرا می‌شود. چون break ندارد، به `case 1` سقوط می‌کند و آنجا break می‌خورد.

تمرین درس ۳

برگشت ← صفحهٔ ۲۷

۱. `10` (یک بار اجرا می‌شود، با اینکه شرط از اول نادرست است)

۲. `34 103` (`34 = 1 + ⌊99/3⌋` ؛ آخرین مقداری که شرط را نقض کرد ۱۰۳ است)

۳. `1 2 3 | 4`

۴. `43210`

۵. `10` (`1 + 2 + 3 + 4`)

تمرین درس ۴

برگشت ← صفحهٔ ۲۹

۱. `3` . break فقط حلقهٔ درونی را می‌شکند، پس در هر دور بیرونی فقط `j = 0` شمرده می‌شود.

۲. `37` . جمع ۱ تا ۱۰ منهای مضرب‌های ۳: `55 - (3+6+9) = 37` . چون `i++` قبل از continue است، حلقه بی‌نهایت نمی‌شود.

آزمون فصل ۲ — کلید و راه‌حل

برگشت ← صفحهٔ ۳۰

سؤال ۱	سؤال ۲	سؤال ۳	سؤال ۴	سؤال ۵	سؤال ۶	سؤال ۷	سؤال ۸	سؤال ۹	سؤال ۱۰
۱	۳	۲	۲	۲	۲	۳	۳	۱	۱

۱. else مال `if (y > 0)` است. شرط بیرونی (`5 > 10`) نادرست است، پس هیچ‌چیز اجرا نمی‌شود و x همان **۵** می‌ماند.

۲. از `case 3` وارد می‌شود و تا آخر سقوط می‌کند: `3 + 5 + 100 = 108` .

۳. do-while یک بار اجرا می‌شود: `s = 0` و `i = 2` . بعد شرط `2 < 0` نادرست است.

۴. i مقدارهای ۱، ۳ و ۹ را می‌گیرد؛ ۲۷ دیگر از ۲۰ بیشتر است. پس **۳** تکرار.

۵. `0 + 1 + 3 + 4 + 5 = 13` (۲ رد می‌شود و با ۶ خارج می‌شویم).

۶. مقایسه‌ها: `10>7` (i=۹)، `9>7` (i=۸)، `8>7` (i=۷)، `7>7` نادرست ولی کاهش باز هم انجام می‌شود، پس **i=۶**.

۷. زوج‌های `j < i` با `i + j` فرد: `(1,0)` ، `(2,1)` ، `(3,0)` ، `(3,2)` ، `(4,1)` ، `(4,3)` . یعنی **۶**.

۸. `-4 % 3 = -1` (علامت عدد اول)، پس `case -1` اجرا می‌شود و جواب **۳**.

۹. دورها: `(0,10)` ، `(2,8)` ، `(4,6)` شمرده می‌شوند و با `(6,4)` شرط نقض می‌شود. خروجی `3 6 4` .

۱۰. برای `i=1` : با j=۱ رد و با j=۲ خارج. برای `i=2` : فقط ۲۱. برای `i=3` : `31 + 32 = 63` . جمع **۸۴**.

فصل ۳

۳ تابع، ارسال پارامتر، حوزهٔ متغیر، static و ماکرو

۴ درس | ۱۰ تست تألیفی در آزمون فصل

💡 **چرا این فصل؟** تست‌های تابع تقریباً همیشه یکی از این چهار تله را دارند: «مقدار در تابع عوض شد ولی بیرون نه»، متغیر `static` که مقدارش می‌ماند، متغیر هم‌نام محلی و سراسری، و ماکروی بدون پرانتز.

✅ **اعتبار** خروجی همهٔ کدها با `gcc` اجرا و چک شده است. (مثال‌های تکمیلی: `کد/مثال-ها.c`)

⏱️ **زمان مطالعه** یک بلوک ۹۰ دقیقه‌ای.

نقشهٔ فصل: ۱ تعریف و فراخوانی تابع | ۲ ارسال پارامتر | ۳ حوزه (scope) و طول عمر | ۴ ماکرو (#define)
📝 آزمون فصل

درس ۱ تعریف و فراخوانی تابع

از صفر: تابع مثل یک دستگاه

تابع یک **دستگاه** است: چیزی به آن می‌دهی (پارامتر یا ورودی)، کاری انجام می‌دهد و ممکن است یک جواب پس بدهد (مقدار برگشتی).

```
int sq(int n) {        // نوع جواب: int · اسم: sq · یک ورودی: int به اسم n
    return n * n;      // جواب را پس می‌دهد و تابع تمام می‌شود
}
```

وقتی تابع صدا زده می‌شود چه اتفاقی می‌افتد؟

۱. اجرا از خط فعلی **می‌پرد** به اول تابع.

۲. پارامتر n یک **جعبهٔ تازه** است که مقدار ورودی در آن کپی می‌شود.

۳. بدنهٔ تابع اجرا می‌شود تا به `return` برسد.

۴. اجرا به همان جای قبلی **برمی‌گردد** و مقدار برگشتی **جای خود فراخوانی** می‌نشیند.

✏️ **مثال (ردگیری)**

```
int a = sq(3) + sq(2);
printf("%d", a);
```

`sq(3)` اجرا می‌شود و ۹ برمی‌گرداند؛ `sq(2)` اجرا می‌شود و ۴ برمی‌گرداند. عبارت می‌شود `9 + 4` ⇐ خروجی `13`.

اعلان و تعریف

```c
int add(int a, int b);          // اعلان (prototype): «چنین تابعی وجود دارد»
int add(int a, int b) {         // تعریف: بدنهٔ واقعی
    return a + b;
}
```

اعلان فقط به کامپایلر می‌گوید تابع چه ورودی و خروجی‌ای دارد، تا بتوانی آن را قبل از تعریفش صدا بزنی.

سه نکته

- نوع `void` یعنی تابع مقدار برنمی‌گرداند؛ فقط کاری انجام می‌دهد (مثلاً چاپ). مثال: `void hello(int k) { printf("hi%d ", k); }` با `hello(1); hello(2);` چاپ می‌کند `hi1 hi2`.
- **ترتیب ارزیابی آرگومان‌ها در C مشخص نیست.** پس در `f(i++, i++)` نتیجه قطعی نیست و تست درست آن را نمی‌پرسد.
- `return` تابع را **فوراً** تمام می‌کند؛ کدهای بعد از آن اجرا نمی‌شوند.

✏️ مثال

```c
int sign(int x) {
    if (x < 0)  return -1;    // با x = -7 همین‌جا تمام می‌شود
    if (x == 0) return 0;
    return 1;
}
```

`sign(-7) = -1`، `sign(0) = 0`، `sign(4) = 1`. با `x = -7`، دو خط بعدی اصلاً اجرا نمی‌شوند.

درس ۲ ارسال پارامتر

از صفر: فتوکپی یا آدرس خانه؟

وقتی متغیری را به تابع می‌دهی، تابع یک **فتوکپی** از مقدارش می‌گیرد. هر خط‌خوردگی روی فتوکپی، روی اصل سند اثری ندارد. اسم این روش **ارسال با مقدار** است و در C **همه‌چیز** همین‌طور فرستاده می‌شود.

✏️ مثال (ردگیری)

```c
void twice(int v) { v = v * 2; printf("in:%d ", v); }
int x = 4;
twice(x);
printf("out:%d", x);
```

خروجی `in:8 out:4`. داخل تابع، v (کپی) هشت شد؛ x اصلی همان ۴ ماند.

اگر بخواهی تابع خودِ متغیر را عوض کند، باید **آدرس خانه‌اش** را بدهی، نه فتوکپی‌اش. آدرس را با `&x` می‌دهی و تابع با `*` به خانه سر می‌زند (جزئیات در فصل ۵).

```
void swap_v(int a, int b) { int t = a; a = b; b = t; }      // کار نمی‌کند
void swap_p(int *a, int *b) { int t = *a; *a = *b; *b = t; } // کار می‌کند
```

با `x = 1` و `y = 2`، بعد از `swap_v(x, y)` همان `1 2` می‌ماند (فقط کپی‌ها جابه‌جا شدند). بعد از `swap_p(&x, &y)` می‌شود `2 1`.

> 💡 **قاعده**
>
> برای تغییر متغیر بیرونی، آدرسش را بفرست (اشاره‌گر).

آرایه استثنای ظاهری است

وقتی آرایه را به تابع می‌دهی، **آدرس اولین عضو** فرستاده می‌شود، نه کپی کل آرایه. پس تغییر عضوها در تابع، آرایهٔ اصلی را عوض می‌کند.

```
void set0(int a[]) { a[0] = 9; }
int v[3] = {1, 2, 3};  set0(v);   // حالا v[0] == 9
```

> ⚠️ **تلهٔ تستی**
>
> داخل تابع، `sizeof(a)` اندازهٔ **اشاره‌گر** است (۸ بایت در سیستم ۶۴ بیتی)، نه اندازهٔ آرایه. بیرون تابع، `sizeof(v)` برای `int v[10]` برابر ۴۰ است.

✍️ تمرین درس ۲ — اول خودت حل کن، بعد پاسخ‌نامه

```
void g(int x)  { x = x * 2; }
void h(int *p) { *p = *p * 2; }
int a = 5;  g(a);  printf("%d ", a);  h(&a);  printf("%d", a);
```

🔑 پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: **صفحهٔ ۴۴**

درس ۳ حوزه (scope) و طول عمر

از صفر: دو سؤال دربارهٔ هر متغیر

۱. **حوزه:** این متغیر از **کجای** کد دیده می‌شود؟

۲. **طول عمر:** این متغیر **تا کی** زنده است و مقدارش را نگه می‌دارد؟

تشبیه:

- **متغیر محلی معمولی** مثل نوشتن روی تختهٔ کلاس است: هر بار کلاس (تابع) شروع می‌شود تخته تمیز است و آخر کلاس پاک می‌شود.
- **متغیر محلی static** مثل دفتر شخصی معلم در همان کلاس است: فقط در همان کلاس دیده می‌شود، ولی بین جلسه‌ها پاک نمی‌شود.
- **متغیر سراسری** مثل تابلوی اعلانات راهرو است: همه از همه‌جا می‌بینندش و تا آخر برنامه می‌ماند.

نوع متغیر	دیده می‌شود در	طول عمر	مقدار اولیهٔ پیش‌فرض
محلی معمولی (auto)	همان بلوک `{ }`	هر بار ورود به بلوک ساخته و در خروج نابود می‌شود	**نامعلوم (زباله)**
محلی `static`	همان بلوک	**کل اجرای برنامه**؛ مقدارش بین فراخوانی‌ها می‌ماند	۰
سراسری	کل فایل	کل اجرای برنامه	۰

✏️ **مثال (معمولی در برابر static)**

```
void f(void) { int a = 0; static int s = 0; a++; s++; printf("%d%d ", a, s); }
f(); f();
```

خروجی `11 12` . در هر فراخوانی a از صفر ساخته می‌شود و ۱ می‌شود؛ ولی s از فراخوانی قبل مقدارش را نگه داشته و در دومی ۲ می‌شود.

✏️ **مثال (مقدار اولیهٔ پیش‌فرض)**

```
int g;                      // 0 ⇐ سراسری
int main(void) {
    static int s;           // static ⇐ 0
    int a;                  // محلی ⇐ زباله؛ چاپش قابل پیش‌بینی نیست
    printf("%d %d", g, s);  // 0 0
}
```

دو قاعدهٔ کلیدی

۱. **مقداردهی اولیهٔ `static` فقط یک بار انجام می‌شود.** خط `static int c = 0;` در فراخوانی دوم دوباره c را صفر نمی‌کند.

۲. **پوشاندن (shadowing):** متغیر محلی هم‌نام، متغیر سراسری یا بیرونی را در آن بلوک **پنهان** می‌کند. ولی تابعی که بیرون تعریف شده، همیشه سراسری را می‌بیند.

✏️ مثال

```
int x = 5;                              // سراسری
void hx(void) { printf("%d ", x); }     // همیشه سراسری را می‌بیند
int main(void) {
    int x = 10;
    hx();                               // 5
    { int x = 20; printf("%d ", x); }   // 20
    printf("%d", x);                    // 10
}
```

خروجی `5 20 10` . hx فقط x سراسری را می‌شناسد؛ x های داخل main برای او وجود ندارند.

💡 روش

وقتی متغیری هم‌نام چند جا تعریف شده، برای هر استفاده بپرس «نزدیک‌ترین تعریف در همین بلوک یا بلوک‌های بیرونی‌اش کدام است؟»

✍️ تمرین درس ۳ — اول خودت حل کن، بعد پاسخ‌نامه

```
// 1
int counter(void) { static int c = 0; c++; return c; }
counter();  counter();  printf("%d", counter());
// 2
int fa(void) { int a = 0; static int s = 0; a++; s++; return a*10 + s; }
fa();  fa();  printf("%d", fa());
```

🔑 پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: **صفحهٔ ۴۴**

درس ۴ ماکرو (#define)

از صفر: «جستجو و جایگزینی» قبل از کامپایل

ماکرو تابع **نیست**. قبل از کامپایل، یک برنامهٔ کمکی (پیش‌پردازنده) متن کد را می‌خواند و هر جا اسم ماکرو را دید، **متنش را عیناً جایگزین می‌کند**؛ دقیقاً مثل «Find and Replace» در Word. هیچ محاسبه‌ای در این مرحله انجام نمی‌شود و اولویت عملگرها هم رعایت نمی‌شود.

✏️ **مثال**

با `#define SQ(x) x*x` ، عبارت `SQ(2+1)` به‌صورت متنی می‌شود `2+1*2+1` و چون ضرب اولویت دارد: `2 + 2 + 1 = 5`، نه ۹.

ماکرو	`SQ(1+2)`	`100 / SQ(5)`
`#define SQ(x) x*x`	`1+2*1+2 = 5`	`100/5*5 = 100`
`#define SQ(x) ((x)*(x))`	`9`	`4`

💡 **روش**

ماکرو را روی کاغذ **کلمه‌به‌کلمه باز کن**، بعد با جدول اولویت فصل ۱ حساب کن.

💡 **قاعده**

هر پارامتر را در پرانتز بگذار و کل عبارت را هم در پرانتز.

عملگر شرطی `? :`

در ماکروها زیاد می‌آید: `شرط ? الف : ب` یعنی «اگر شرط درست است الف، وگرنه ب». مثل یک if کوچک که مقدار برمی‌گرداند.

✏️ **مثال**

با `m = 7 : m > 5 ? 1 : 0` ⇐ ۱؛ `m % 2 == 0 ? m : -m` ⇐ `-7`.

✏️ **مثال (پرانتز بیرونی)**

```
#define MAX(a,b)    ((a) > (b) ? (a) : (b))
#define BADMAX(a,b)  (a) > (b) ? (a) : (b)
2 * MAX(2, 3)      // 6
2 * BADMAX(2, 3)   // 2
```

دومی باز می‌شود به `2 * (2) > (3) ? (2) : (3)` ، یعنی `(4 > 3) ? 2 : 3` که ۲ است.

⚠️ **تلهٔ تستی**

حتی ماکروی درست هم با `++` خطرناک است، چون آرگومان ممکن است **دو بار** ارزیابی شود: `MAX(i++, 0)` باز می‌شود به `((i++) > (0) ? (i++) : (0))` و i ممکن است دو بار زیاد شود (سؤال ۱۰ آزمون).

📝 آزمون فصل ۳

تست‌های تألیفی (خروجی همه با gcc اجرا و تأیید شده)

۱۰ سؤال | ⏱️ زمان: ۱۵ دقیقه | 🔑 کلید و راه‌حل: صفحهٔ ۴۴

این درس در کنکور ارشد کامپیوتر سؤال سابقه ندارد. پس این ۱۰ تست **تألیفی** هستند و خروجی همه با `gcc` تأیید شده است.

خروجی هر کد چیست؟

✔️ مثل جلسهٔ کنکور: زمان بگیر، روی هیچ سؤالی بیشتر از ۴ دقیقه نمان، و در مربع کنار سؤال‌های غلط یا حدسی علامت بزن تا در دور دوم فقط همان‌ها را حل کنی.

۱ تألیفی — غلط یا حدسی؟ ☐

```
int f(int n) { static int s = 1; s *= n; return s; }
// main:
f(2); f(3); printf("%d", f(4));
```

۱) 4 ۲) 8 ۳) 24 ۴) 12

۲ تألیفی — غلط یا حدسی؟ ☐

```
void g(int x) { x = x * 2; }
// main:
int a = 5; g(a); g(a); printf("%d", a);
```

۱) 5 ۲) 10 ۳) 20 ۴) مقدار نامعلوم

۳ تألیفی — غلط یا حدسی؟ ☐

```
void g(int *p) { *p = *p * 2; }
// main:
int a = 5; g(&a); g(&a); printf("%d", a);
```

۱) 5 ۲) 10 ۳) 20 ۴) آدرس a

۴ تألیفی — غلط یا حدسی؟ ☐

```
#define DOUBLE(x) x + x
printf("%d", 3 * DOUBLE(4));
```

۱) 24 ۲) 16 ۳) 14 ۴) 12

۵ تألیفی — غلط یا حدسی؟

```
int x = 1;
int f(void) { return x * 10; }
int main(void) { int x = 2; printf("%d", f() + x); }
```

۱) 12 ۲) 22 ۳) 11 ۴) 21

۶ تألیفی — غلط یا حدسی؟

```
void z(int a[], int n) { for (int i = 0; i < n; i++) a[i] = a[i] * i; }
// main:
int v[4] = {5, 5, 5, 5}; z(v, 4); printf("%d %d", v[0], v[3]);
```

۱) 5 5 ۲) 0 15 ۳) 5 15 ۴) 0 20

۷ تألیفی — غلط یا حدسی؟

```
int t(void) { static int k = 10; return k--; }
// main:
int s = 0; for (int i = 0; i < 3; i++) s += t(); printf("%d", s);
```

۱) 30 ۲) 27 ۳) 24 ۴) 9

۸ تألیفی — غلط یا حدسی؟

```
int g = 0;
int h(int v) { g += v; return g; }
// main:
int a = h(3); int b = h(4); printf("%d %d %d", a, b, g);
```

۱) 3 4 7 ۲) 3 7 7 ۳) 3 7 0 ۴) 7 7 7

۹ تألیفی — غلط یا حدسی؟

```
for (int i = 0; i < 3; i++) {
    int a = 0; static int b = 0;
    a += i; b += i;
    if (i == 2) printf("%d %d", a, b);
}
```

۱) 2 2 ۲) 3 3 ۳) 2 3 ۴) 3 2

۱۰ تألیفی

غلط یا حدسی؟

```c
#define MAX(a,b) ((a) > (b) ? (a) : (b))
int x = 5, y = 3;
int m = MAX(x++, y);
printf("%d %d", m, x);
```

۱) 5 6
۲) 6 7
۳) 5 7
۴) 6 6

🗂️ کارت خلاصهٔ فصل ۳

برای مرور سریع، شب قبل از آزمون

- ✔ C همه‌چیز را با مقدار می‌فرستد. برای تغییر متغیر بیرونی، آدرسش را بفرست.
- ✔ آرایه به‌صورت آدرس فرستاده می‌شود، پس تغییر عضوها بیرون دیده می‌شود. `sizeof` آرایه داخل تابع اندازهٔ اشاره‌گر است.
- ✔ محلی معمولی هر بار از نو ساخته می‌شود. `static` یک بار مقداردهی می‌شود و مقدارش می‌ماند. سراسری و static پیش‌فرض صفرند.
- ✔ متغیر محلی هم‌نام، سراسری را می‌پوشاند؛ ولی تابع‌های دیگر سراسری را می‌بینند.
- ✔ ماکرو جایگزینی متنی است. بدون پرانتز اولویت به هم می‌ریزد و با `++` آرگومان ممکن است دو بار اجرا شود.

پاسخ‌نامهٔ فصل ۳

تا تمرین‌ها و آزمون فصل را خودت حل نکرده‌ای، این صفحه‌ها را نخوان.

تمرین درس ۲

برگشت ← صفحهٔ ۳۷

`5 10` . تابع g روی کپی کار می‌کند و تابع h از طریق آدرس، خود a را دو برابر می‌کند.

تمرین درس ۳

برگشت ← صفحهٔ ۳۹

۱. `3`

۲. `13` . متغیر a هر بار از صفر شروع می‌شود و ۱ می‌شود؛ s در فراخوانی سوم ۳ است.

آزمون فصل ۳ — کلید و راه‌حل

برگشت ← صفحهٔ ۴۱

سؤال ۱	سؤال ۲	سؤال ۳	سؤال ۴	سؤال ۵	سؤال ۶	سؤال ۷	سؤال ۸	سؤال ۹	سؤال ۱۰
۳	۱	۳	۲	۱	۲	۲	۲	۳	۲

۱. s مقدارش را نگه می‌دارد: `1·2·3·4 = 24` .

۲. g روی کپی کار می‌کند، پس a همان **۵** است.

۳. از طریق اشاره‌گر دو بار دو برابر می‌شود: `5 → 10 → 20` .

۴. جایگزینی متنی: `3 * 4 + 4 = 16` .

۵. f متغیر سراسری x (یعنی ۱) را می‌بیند: `10 + 2 = 12` .

۶. `a[i] = 5·i` ، پس `v[0] = 0` و `v[3] = 15` .

۷. `k--` اول مقدار قدیمی را برمی‌گرداند: `10 + 9 + 8 = 27` .

۸. `a = 3` ، بعد `g = 7` پس `b = 7` ، و در آخر `g = 7` .

۹. a هر دور از صفر شروع می‌شود و در دور آخر ۲ است. b جمع می‌شود: `0 + 1 + 2 = 3` .

۱۰. ماکرو باز می‌شود به `((x++) > (y) ? (x++) : (y))` . اول `x++` مقدار ۵ را می‌دهد (و x می‌شود ۶). چون `5 > 3`، دوباره `x++` ارزیابی می‌شود که ۶ را برمی‌گرداند و x را ۷ می‌کند. نتیجه `m = 6` و `x = 7` . (بعد از عملوند اول `?:` نقطهٔ ترتیب هست، پس رفتار تعریف‌شده است.)

فصل ۴

بازگشت و ردگیری فراخوانی

۴

۲ درس ۱۰ تست تألیفی در آزمون فصل

💡 **چرا این فصل؟** تست‌های بازگشت دو نوع‌اند: «خروجی چیست؟» (ترتیب چاپ قبل یا بعد از فراخوانی) و «چند بار فراخوانی می‌شود؟». هر دو با یک روش حل می‌شوند: **درخت فراخوانی بکش.** این فصل پایهٔ روابط بازگشتی (ساختمان داده فصل ۱) و برنامه‌نویسی پویا هم هست.

✅ **اعتبار** خروجی همهٔ کدها با `gcc` اجرا و چک شده است. (مثال‌های تکمیلی: `کد/مثال-ها.c`)

⏱️ **زمان مطالعه** یک بلوک ۹۰ دقیقه‌ای.

نقشهٔ فصل: ۱ ساختار تابع بازگشتی و ترتیب چاپ | ۲ شمردن فراخوانی‌ها و توابع کلاسیک | 📝 آزمون فصل

درس ۱ ساختار تابع بازگشتی و ترتیب چاپ

از صفر: صف سینما

در صف طولانی سینما ایستاده‌ای و می‌خواهی بدانی نفر چندمی. جلو را نمی‌بینی. از نفر جلویی می‌پرسی «تو نفر چندمی؟». او هم نمی‌داند و از جلویی خودش می‌پرسد … تا به **نفر اول** می‌رسد که بدون پرسیدن می‌داند: «من اولم». بعد جواب‌ها برمی‌گردند: «من دومم»، «من سومم»، … تا به تو برسد.

این دقیقاً **بازگشت** است: تابعی که برای حل مسئله، **خودش را روی نسخهٔ کوچک‌تر همان مسئله** صدا می‌زند.

- **حالت پایه:** جایی که بدون پرسیدن جواب معلوم است (نفر اول). بدون آن، پرسیدن تا ابد ادامه پیدا می‌کند و برنامه از کار می‌افتد.
- **قدم بازگشتی:** مسئله را کوچک‌تر کن، از خودت بپرس، و جواب را کامل کن (یکی به جواب جلویی اضافه کن).

یک تابع بازگشتی ساده

```c
int s(int n) {                  // جمع ۱ تا n
    if (n == 0) return 0;       // حالت پایه
    return n + s(n - 1);        // قدم بازگشتی
}
```

✏️ **مثال (ردگیری s(۴))**

```
s(4) = 4 + s(3)
          s(3) = 3 + s(2)
                    s(2) = 2 + s(1)
                              s(1) = 1 + s(0)
                                        s(0) = 0   ← پایه
```

حالا برگشت از پایین به بالا: `s(1) = 1` ، `s(2) = 3` ، `s(3) = 6` ، `s(4) = 10` .

هر فراخوانی جعبه‌های خودش را دارد

هر بار که تابع صدا زده می‌شود، یک **قاب** تازه روی **پشتهٔ فراخوانی** ساخته می‌شود با n و متغیرهای محلی **مخصوص خودش**. n در `s(4)` و n در `s(3)` دو جعبهٔ جدا هستند. وقتی یک فراخوانی تمام می‌شود، قابش برداشته می‌شود و اجرا دقیقاً از همان نقطه‌ای که در قاب قبلی متوقف شده بود ادامه پیدا می‌کند.

✏️ **مثال (پشتهٔ فراخوانی برای fact(۳))**

با `fact(n) = n × fact(n-1)` و `fact(1) = 1` :

- رفت: `fact(3)` صبر می‌کند تا `fact(2)` تمام شود؛ `fact(2)` صبر می‌کند تا `fact(1)` .
- `fact(1)` حالت پایه است و ۱ برمی‌گرداند.
- برگشت: `fact(2) = 2 × 1 = 2` ، بعد `fact(3) = 3 × 2 = 6` . در اوج، ۳ قاب روی پشته بود و هر کدام n خودش را داشت.

ترتیب چاپ: رفت یا برگشت؟

تست‌های این فصل بیشتر می‌پرسند «خروجی چیست؟». کلید جواب، جای printf نسبت به فراخوانی بازگشتی است.

💡 **قاعدهٔ طلایی ترتیب چاپ**

- چاپ **قبل** از فراخوانی ⇐ در مسیر **رفت** چاپ می‌شود (ترتیب نزولی).
- چاپ **بعد** از فراخوانی ⇐ در مسیر **برگشت** چاپ می‌شود (ترتیب صعودی).

```
void p(int n) { if (n == 0) return; printf("%d ", n); p(n-1); }   // p(3): 3 2 1
void q(int n) { if (n == 0) return; q(n-1); printf("%d ", n); }   // q(3): 1 2 3
void r(int n) { if (n <= 0) return; printf("%d ", n); r(n-2); printf("%d ", n); }
// r(5): 5 3 1 1 3 5   (یک بار در رفت، یک بار در برگشت)
```

✏️ مثال (چرا q صعودی چاپ می‌کند؟)

`q(3)` قبل از چاپ کردن، `q(2)` را صدا می‌زند و منتظر می‌ماند؛ `q(2)` منتظر `q(1)` ؛ `q(1)` منتظر `q(0)` که فوراً برمی‌گردد. حالا `q(1)` به خط چاپ می‌رسد ⇐ «۱»، بعد برمی‌گردد و `q(2)` چاپ می‌کند ⇐ «۲»، و آخر `q(3)` ⇐ «۳».

کسی که اول صدا زده شده، آخر از همه چاپ می‌کند.

💡 روش

درخت فراخوانی را بکش (هر فراخوانی یک گره، فرزندانش فراخوانی‌هایی که انجام می‌دهد). کنار هر گره بنویس printf «قبل» از فرزندان است یا «بعد». بعد درخت را از ریشه، چپ به راست پیمایش کن و چاپ‌ها را به همان ترتیب بنویس.

✍️ تمرین درس ۱

اول خودت حل کن، بعد پاسخ‌نامه

```
// 1
int g(int n) { if (n == 0) return 0; return n % 10 + g(n / 10); }      // g(1234)?
// 2
void b(int n) { if (n > 1) b(n / 2); printf("%d", n % 2); }             // b(13)?
```

🔑 پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: صفحهٔ ۵۳

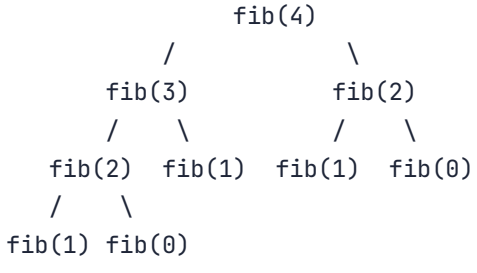
درس ۲ شمردن فراخوانی‌ها و توابع کلاسیک

از صفر: چرا بعضی توابع بازگشتی منفجر می‌شوند؟

اگر هر فراخوانی **یک بار** خودش را صدا بزند (مثل s و fact)، فراخوانی‌ها یک زنجیرند و تعدادشان حدود n است. اگر هر فراخوانی **دو بار** خودش را صدا بزند، درخت فراخوانی دوشاخه می‌شود و تعداد گره‌ها می‌تواند **نمایی** رشد کند.

✏️ مثال (درخت fib(۴))

با `fib(n) = fib(n-1) + fib(n-2)` و پایه‌های `fib(0)` و `fib(1)`:



۹ گره، یعنی ۹ فراخوانی. `fib(2)` دو بار از نو حساب شده است. برای n های بزرگ این تکرار بیهوده انفجاری می‌شود (همین مشکل را در ساختمان داده با «برنامه‌نویسی پویا» حل می‌کنند).

💡 روش شمردن

رابطهٔ «تعداد فراخوانی» را بنویس: `C(n) = 1 + C(زیرمسئله‌ها)` ؛ «۱» برای خود فراخوانی. مثلاً برای فیبوناچی `C(n) = 1 + C(n-1) + C(n-2)` و `C(0) = C(1) = 1`.

تابع	تعداد فراخوانی یا هزینه
فاکتوریل `f(n) = n·f(n-1)`	`O(n)`، n فراخوانی
فیبوناچی ساده `fib(n) = fib(n-1) + fib(n-2)`	فراخوانی (نمایی). مثلاً `fib(5)` : ۱۵ و `fib(6)` : ۲۵ `2·F(n+1) - 1`
هانوی	`2ⁿ - 1` حرکت
ب.م.م اقلیدسی `gcd(b, a % b)`	`O(log min(a,b))`
توان سریع `t = pw(x, n/2)` و بعد `t·t`	`O(log n)` ضرب
`h(n) = h(n/2) + h(n/2)` (دو فراخوانی **جدا**)	`2n - 1` فراخوانی برای n توان ۲

✏️ مثال برای هر سطر جدول

- **فیبوناچی:** `C(2) = 1 + 1 + 1 = 3`، `C(3) = 1 + 3 + 1 = 5`، `C(4) = 1 + 5 + 3 = 9`، `C(5) =` 15. چک با فرمول: `2·F(5) - 1 = 2·5 - 1 = 9` برای `fib(4)` ✓ (همان ۹ گره درخت بالا).
- **دو فراخوانی جدا:** `h(4)` ⇐ دو `h(2)` ⇐ چهار `h(1)`. جمع `1 + 2 + 4 = 7 = 2·4 - 1` ✓.
- **ب.م.م:** `gcd(48, 18)` ⇐ `gcd(18, 12)` ⇐ `gcd(12, 6)` ⇐ `gcd(6, 0)` : ۴ فراخوانی.
- **توان سریع:** `pw(2, 10)` ⇐ `pw(2, 5)` ⇐ `pw(2, 2)` ⇐ `pw(2, 1)` ⇐ `pw(2, 0)` : ۵ فراخوانی و جواب ۱۰۲۴.
- **هانوی:** با ۳ دیسک، `2³ - 1 = 7` حرکت.

⚠️ تلهٔ تستی

`pw(x, n/2) * pw(x, n/2)` (دو بار صدا زدن) با `t = pw(x, n/2); t * t` (یک بار) فرق دارد. اولی درخت دوشاخه می‌سازد و `O(n)` است؛ دومی زنجیر است و `O(log n)`.

متغیر static در بازگشت

گفتیم هر قاب جعبه‌های **خودش** را دارد. استثنا: متغیر `static` فقط **یک** جعبه است که بین **همهٔ** فراخوانی‌ها مشترک است (فصل ۳).

✏️ مثال

```
int fs(int n) { static int c = 0; c++; if (n > 0) fs(n - 1); return c; }
printf("%d", fs(3));    // 4
```

`fs(3)` ، `fs(2)` ، `fs(1)` و `fs(0)` هر کدام همان یک c را زیاد می‌کنند. وقتی `fs(3)` به return می‌رسد، c برابر ۴ است. اگر c معمولی (غیر static) بود، جواب ۱ می‌شد.

📝 آزمون فصل ۴

تست‌های تألیفی (خروجی همه با gcc اجرا و تأیید شده)

۱۰ سؤال | ⏱️ زمان: ۱۵ دقیقه | 🔑 کلید و راه‌حل: صفحهٔ ۵۳

این درس در کنکور ارشد کامپیوتر سؤال سابقه ندارد. پس این ۱۰ تست **تألیفی** هستند و خروجی همه با `gcc` تأیید شده است.

✔️ مثل جلسهٔ کنکور: زمان بگیر، روی هیچ سؤالی بیشتر از ۴ دقیقه نمان، و در مربع کنار سؤال‌های غلط یا حدسی علامت بزن تا در دور دوم فقط همان‌ها را حل کنی.

۱ تألیفی — غلط یا حدسی؟ ☐

`int f(int n) { if (n <= 0) return 0; return n + f(n - 2); }` مقدار `f(7)`؟

۱) ۱۶
۲) ۲۸
۳) ۱۲
۴) ۹

۲ تألیفی — غلط یا حدسی؟ ☐

خروجی `p(3)`؟

```
void p(int n) { if (n > 0) { p(n - 1); printf("%d", n); p(n - 1); } }
```

۱) 123
۲) 1213121
۳) 3212123
۴) 1121312

۳ تألیفی — غلط یا حدسی؟ ☐

مقدار برگشتی و تعداد کل فراخوانی‌های `fib(6)`؟

```
int c = 0;
int fib(int n) { c++; if (n < 2) return n; return fib(n-1) + fib(n-2); }
```

۱) 8 25
۲) 8 15
۳) 13 25
۴) 8 13

۴ تألیفی — غلط یا حدسی؟ ☐

```
int g(int x) { if (x > 100) return x - 10; return g(g(x + 11)); }
```

مقدار `g(95)`؟

۱) ۹۵
۲) ۹۱
۳) ۱۰۱
۴) ۸۵

۵ تألیفی

غلط یا حدسی؟

```
int s(int n) { static int k = 0; k += n; if (n > 1) s(n - 1); return k; }
```

مقدار `s(4)`؟

۱) ۴
۲) ۷
۳) ۱۰
۴) ۲۴

۶ تألیفی

غلط یا حدسی؟

```
int f(int n) { if (n == 1) return 1; if (n % 2 == 0) return f(n / 2); return f(3*n + 1) + 1; }
```

مقدار `f(6)`؟

۱) ۱
۲) ۲
۳) ۳
۴) ۸

۷ تألیفی

غلط یا حدسی؟

خروجی `r("abc")`؟

```
void r(const char *s) { if (*s) { r(s + 1); putchar(*s); } }
```

۱) `abc`
۲) `cba`
۳) `a`
۴) `c`

۸ تألیفی

غلط یا حدسی؟

```
int m(int a, int b) {
    if (b == 0) return 0;
    if (b % 2 == 0) return m(a + a, b / 2);
    return m(a + a, b / 2) + a;
}
```

مقدار `m(7, 6)`؟

۱) ۱۳
۲) ۴۲
۳) ۴۹
۴) ۲۸

۹ تألیفی

غلط یا حدسی؟ ☐

مقدار برگشتی `h(8)` و تعداد کل فراخوانی‌ها؟

```
int c = 0;
int h(int n) { c++; if (n ⩽ 1) return 1; return h(n/2) + h(n/2); }
```

۱) `8 15`
۲) `4 7`
۳) `8 8`
۴) `1 4`

۱۰ تألیفی

غلط یا حدسی؟ ☐

```
int rev(int n, int acc) { return n == 0 ? acc : rev(n / 10, acc * 10 + n % 10); }
```

مقدار `rev(1203, 0)`؟

۱) ۳۰۲۱
۲) ۳۲۱
۳) ۱۲۰۳
۴) ۲۱۰۳

✂️

🗂️ کارت خلاصهٔ فصل ۴

برای مرور سریع، شب قبل از آزمون

- ✔ چاپ قبل از فراخوانی: رفت (نزولی). چاپ بعد از فراخوانی: برگشت (صعودی). هر دو: الگوی آینه‌ای.
- ✔ تعداد فراخوانی را با `C(n) = 1 + Σ C(زیرمسئله)` بشمار. فیبوناچی: `2F(n+1) - 1`.
- ✔ دو بار صدا زدن زیرمسئلهٔ یکسان، هزینه را چند برابر می‌کند. یک بار صدا زدن و ذخیره کردن، `O(log n)` می‌دهد.
- ✔ static بین همهٔ قاب‌ها مشترک است. تابع ۹۱: همیشه ۹۱ برای `x ≤ 101`.

پاسخ‌نامهٔ فصل ۴

تا تمرین‌ها و آزمون فصل را خودت حل نکرده‌ای، این صفحه‌ها را نخوان.

تمرین درس ۱

برگشت ← صفحهٔ ۴۷

۱. `10` (جمع ارقام)

۲. `1101` . نمایش دودویی ۱۳ است: رقم‌ها در برگشت چاپ می‌شوند، پس پرارزش‌ترین رقم اول می‌آید.

آزمون فصل ۴ — کلید و راه‌حل

برگشت ← صفحهٔ ۵۰

سؤال ۱	سؤال ۲	سؤال ۳	سؤال ۴	سؤال ۵	سؤال ۶	سؤال ۷	سؤال ۸	سؤال ۹	سؤال ۱۰
۱	۲	۱	۲	۳	۳	۲	۲	۱	۱

۱. `7 + 5 + 3 + 1 = 16`

۲. `p(1) = "1"` ، `p(2) = p(1) 2 p(1) = "121"` ، `p(3) = "121" 3 "121" = 1213121`

۳. `fib(6) = 8` و تعداد فراخوانی `2·F(7) - 1 = 2·13 - 1 = 25`

۴. تابع ۹۱ مک‌کارتی: برای هر `x ≤ 101` جواب **۹۱** است.

۵. k مشترک است: `4 + 3 + 2 + 1 = 10` . همهٔ قاب‌ها همان k را برمی‌گردانند.

۶. `f(6) = f(3)` ، `f(3) = f(10) + 1` ، `f(10) = f(5)` ، `f(5) = f(16) + 1` ، و `f(16) = f(8) = f(4) = f(2) = f(1) = 1` . جمع `1 + 1 + 1 = 3` .

۷. چاپ بعد از فراخوانی است، پس رشته وارون می‌شود: `cba` .

۸. ضرب دهقان روسی: `m(a,b) = a·b` ، یعنی `7 × 6 = 42` .

۹. مقدار `h(8) = 8` . تعداد فراخوانی: `C(1) = 1` ، `C(2) = 3` ، `C(4) = 7` ، `C(8) = 15` .

۱۰. ارقام وارون می‌شوند: `3021` .

فصل ۵

اشاره‌گر، آرایه و رشته

۵

۴ درس | ۱۰ تست تألیفی در آزمون فصل

💡 **چرا این فصل؟** سخت‌ترین تست‌های «خروجی کد» در C معمولاً از همین فصل می‌آیند: `*p++` در برابر `(*p)++` ، حساب اشاره‌گر، آرایهٔ دوبعدی، و `sizeof` در برابر `strlen` .

📐 **فرض‌ها** `int` ۴ بایتی و اشاره‌گر ۸ بایتی (سیستم ۶۴ بیتی).

✅ **اعتبار** خروجی همهٔ کدها با `gcc` اجرا و چک شده است. (مثال‌های تکمیلی: `کد/مثال-ها.c`)

⏱️ **زمان مطالعه** یک بلوک ۹۰ دقیقه‌ای.

نقشهٔ فصل: ۱ آرایه از صفر | ۲ اشاره‌گر | ۳ آرایه و حساب اشاره‌گر | ۴ رشته | 📝 آزمون فصل

درس ۱ آرایه از صفر

آرایه یعنی چند جعبهٔ هم‌نوع پشت سر هم

`int a[5];` یعنی ۵ جعبهٔ int که **پشت سر هم** در حافظه چیده شده‌اند و همه با یک اسم (a) و یک شماره (اندیس) صدا زده می‌شوند.

```
int a[5] = {10, 20, 30, 40, 50};
```

اندیس	a[0]	a[1]	a[2]	a[3]	a[4]
مقدار	۱۰	۲۰	۳۰	۴۰	۵۰

- شماره‌گذاری از **صفر** شروع می‌شود. آرایهٔ ۵ تایی اندیس ۰ تا ۴ دارد؛ `a[5]` خارج از آرایه است (رفتار تعریف‌نشده).
- اگر مقدار اولیه کمتر از اندازه باشد، بقیه **صفر** می‌شوند: با `int b[5] = {1, 2};` داریم `b[2] = b[3] = b[4] =` `0`.

✏️ **مثال (پیمایش با حلقه)**

```
int s = 0;
for (int i = 0; i < 5; i++) s += a[i];
printf("%d %d %d", a[0], a[4], s);   // 10 50 150
```

الگوی `for (i = 0; i < n; i++)` دقیقاً همهٔ اندیس‌های ۰ تا `n-1` را می‌بیند.

درس ۲ اشاره‌گر

از صفر: خانه، پلاک، و کاغذی که پلاک رویش نوشته شده

هر متغیر در حافظه یک **خانه** است و هر خانه یک **پلاک** (آدرس) دارد؛ مثلاً خانهٔ x پلاک ۱۰۰۰ دارد و داخلش ۵ است.

- `&x` یعنی «پلاک خانهٔ x» ⇐ ۱۰۰۰.
- **اشاره‌گر** متغیری است که داخلش یک پلاک نوشته شده: `int *p = &x;` ⇐ داخل p عدد ۱۰۰۰ است.
- `*p` یعنی «برو به پلاکی که در p نوشته شده و داخل آن خانه را ببین» ⇐ ۵.

متغیر	پلاک (آدرس)	محتوا
x	۱۰۰۰	۵
p	۲۰۰۰	۱۰۰۰ (پلاک x)

💡 **نتیجهٔ کلیدی**

`*p` **خود x** است، نه کپی آن. پس `*p = 8` یعنی x هشت می‌شود.

✏️ **مثال**

```
int x = 5; int *p = &x; printf("%d ", *p); *p = 8; printf("%d", x);  ⇐ 5 8
```

⚠️ **تلهٔ تستی**

ستاره دو معنی دارد: در **تعریف** (`int *p`) یعنی «p اشاره‌گر است»؛ در **استفاده** (`*p = 8`) یعنی «برو به آدرس داخل p».

جدول عبارت‌ها

عبارت	معنی
`&x`	آدرس x
`*p`	مقداری که p به آن اشاره می‌کند
`int **pp = &p;`	اشاره‌گر به اشاره‌گر؛ `**pp` همان x است
`*p++`	یعنی `*(p++)` : **مقدار فعلی** را بده، بعد **اشاره‌گر** جلو برود
`(*p)++`	**مقدار** را یکی زیاد کن (اشاره‌گر ثابت می‌ماند)؛ مقدار قدیمی را برمی‌گرداند
`++*p`	مقدار را زیاد کن و مقدار **جدید** را برگردان

✏️ **مثال (سه سطر اول جدول)**

```
int x = 5;
int *p = &x;     *p = 7;      // x == 7
int **pp = &p;   **pp = 9;    // x == 9
```

p آدرس x را دارد، پس `*p` خود x است. pp آدرس p را دارد، پس `*pp` همان p و `**pp` همان x است. یعنی دو بار «برو به پلاک».

`*p++` در برابر `(*p)++`

ریشهٔ این تله اولویت است (فصل ۱): `++` پسوندی از `*` قوی‌تر است، پس `*p++` یعنی `*(p++)` و **اشاره‌گر** جلو می‌رود. اگر بخواهی **مقدار** زیاد شود، پرانتز لازم است: `(*p)++` .

✏️ **مثال**

با `a = {10, 20, 30}` و `p = a` (p به ۱۰ اشاره می‌کند)، این سه خط پشت سر هم:

- `x = *p++` ⇐ `x = 10` و p حالا به ۲۰ اشاره می‌کند.
- `y = (*p)++` ⇐ `y = 20` و `a[1]` می‌شود ۲۱ (p تکان نخورد).
- `z = ++*p` ⇐ `z = 22` و `a[1]` می‌شود ۲۲.

درس ۳ آرایه و حساب اشاره‌گر

از صفر: اسم آرایه همان آدرس اولین خانه است

در بیشتر عبارت‌ها، اسم آرایه (a) مثل اشاره‌گری به `a[0]` رفتار می‌کند. چون خانه‌ها پشت سر هم‌اند، با «یکی جلو رفتن» از آدرس اول، به خانهٔ بعدی می‌رسی.

- `a[i]` دقیقاً همان `*(a + i)` است (و حتی `i[a]` هم درست است).
- `p + 1` به **عضو بعدی** اشاره می‌کند، نه بایت بعدی. یعنی آدرس به اندازهٔ `sizeof(*p)` جلو می‌رود.
- **تفریق دو اشاره‌گر** تعداد **عضوهای** بین آن‌هاست: با `p = &a[1]` و `q = &a[4]`، `q - p = 3` .
- نام آرایه را نمی‌شود عوض کرد (`a++` خطاست)، ولی اشاره‌گر را می‌شود.
- `sizeof(a)` برای آرایه، **کل بایت‌ها** است؛ برای اشاره‌گر، ۸. پس تعداد عضوها `sizeof(a)/sizeof(a[0])` است (فقط در همان جایی که آرایه تعریف شده).

✏️ مثال

- اگر `int a[5]` از آدرس ۱۰۰۰ شروع شود، `a + 1` آدرس ۱۰۰۴ است (۴ بایت جلوتر، چون int چهار بایتی است). برای `double d[3]`، `d + 1` هشت بایت جلوتر است.
- `a[3]`، `*(a + 3)` و `3[a]` هر سه یک عضو را می‌دهند.
- `int arr[7];` ⇐ `sizeof(arr) = 28` و `sizeof(arr)/sizeof(arr[0]) = 7`.
- با `a = {10, 20, 30, 40, 50}` و `int *q = a; q++;` : `*q = 20` و `*(q + 2) = 40`.

آرایهٔ دوبعدی

`int b[3][4]` یک جدول ۳ سطر و ۴ ستون است، ولی در حافظه **سطر به سطر پشت سر هم** ذخیره می‌شود: اول ۴ خانهٔ سطر ۰، بعد ۴ خانهٔ سطر ۱، …

- `b[i][j]` همان `*(*(b + i) + j)` است و آدرسش `پایه + (i·4 + j)·4` بایت (از اول جدول، i سطر کامل و j خانه رد شده).
- `sizeof(b) = 48`، `sizeof(b[0]) = 16` (یک سطر).
- اگر `p = &b[0][0]`، آنگاه `*(p + k)` همان عضو kام در ترتیب سطری است.

✏️ مثال

با `int b[3][4] = {{1,2,3,4},{5,6,7,8},{9,10,11,12}}` :

- آدرس `b[2][1]` : `پایه + (2·4 + 1)·4 = پایه + 36` بایت.
- `*(p + 5)` عضو پنجم در ترتیب سطری است (شمارش از صفر): `1, 2, 3, 4, 5, 6` ⇐ **۶** (یعنی `b[1][1]`).

درس ۴ رشته

از صفر: رشته یعنی آرایهٔ char با یک علامت پایان

C نوع جداگانه‌ای برای متن ندارد. رشته یک **آرایه از char** است که آخرش یک کاراکتر ویژه با کد صفر، `'\0'`، آمده تا بگوید «متن همین‌جا تمام شد».

✏️ مثال

`char t[] = "abc";` در حافظه ۴ خانه است: `'a'`، `'b'`، `'c'`، `'\0'`. پس `sizeof(t) = 4`، `strlen(t) = 3`، `t[1] = 'b'` و `t[3]` کاراکتر صفر است.

- `char s[] = "hello";` ⇐ آرایهٔ ۶ بایتی و **قابل تغییر**. `sizeof(s) = 6` و `strlen(s) = 5`.
- `char *t = "hello";` ⇐ اشاره‌گر به رشتهٔ ثابت. `sizeof(t) = 8`. تغییر `t[0]` رفتار تعریف‌نشده است.
- `strlen` تا `'\0'` می‌شمارد (بدون خودش). `sizeof` اندازهٔ آرایه را می‌دهد، هر چقدر هم پر شده باشد.
- `strcmp(a, b)` : منفی اگر a در ترتیب الفبایی قبل از b باشد، صفر اگر برابر باشند، مثبت اگر بعد باشد.
- `printf("%s", p)` از p تا `'\0'` چاپ می‌کند. پس اگر `p = s + 2`، از کاراکتر سوم چاپ می‌شود.

✏️ مثال

- `char u[] = "hello"; u[0] = 'j';` ⇐ `printf("%s %s", u, u + 2)` چاپ می‌کند `jello llo`.
- `char s[10] = "hi";` ⇐ `sizeof(s) = 10` ولی `strlen(s) = 2`.
- `strcmp("abc", "abd") < 0` (در حرف سوم، c قبل از d است)؛ `strcmp("b", "abc") > 0` (حرف اول تعیین می‌کند)؛ `strcmp("ab", "abc") < 0` (پیشوند، قبل است).

✏️ مثال (طول رشته با اشاره‌گر)

```
int len(const char *s) { const char *t = s; while (*t) t++; return t - s; }   // len("kankur") = 6
```

t از اول رشته جلو می‌رود تا به `'\0'` برسد (شرط `*t` صفر می‌شود). فاصلهٔ t از s، همان تعداد کاراکترهاست.

✍️ تمرین فصل ۵

اول خودت حل کن، بعد پاسخ‌نامه

```
// 1
int a[] = {10, 20, 30, 40};  int *p = a;
printf("%d %d %d %d", *p + 1, *(p + 1), p[3], 2[a]);
// 2
int b[3][4] = {{1,2,3,4},{5,6,7,8},{9,10,11,12}};
printf("%d %d", *(*(b + 1) + 2), *(b[2] + 1));
// 3
char s[] = "abcdef";  char *q = s + 2;  printf("%s %c", q, *(q + 2));
```

🔑 پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: **صفحهٔ ۶۲**

📝 آزمون فصل ۵

تست‌های تألیفی (خروجی همه با gcc اجرا و تأیید شده)

۱۰ سؤال | ⏱️ زمان: ۱۵ دقیقه | 🔑 کلید و راه‌حل: صفحهٔ ۶۲

تست‌های **تألیفی**. خروجی همه با `gcc` تأیید شده است.

خروجی هر کد چیست؟

✔️ مثل جلسهٔ کنکور: زمان بگیر، روی هیچ سؤالی بیشتر از ۴ دقیقه نمان، و در مربع کنار سؤال‌های غلط یا حدسی علامت بزن تا در دور دوم فقط همان‌ها را حل کنی.

۱ تألیفی — غلط یا حدسی؟ ☐

```
int a[] = {1, 2, 3, 4, 5};  int *p = a + 1;  p += 2;
printf("%d %ld", *p, (long)(p - a));
```

۱) 4 3　۲) 3 2　۳) 4 12　۴) 5 4

۲ تألیفی — غلط یا حدسی؟ ☐

```
int a[] = {5, 10, 15};  int *p = a;
*p++ += 1;
printf("%d %d", a[0], *p);
```

۱) 5 11　۲) 6 10　۳) 6 6　۴) 5 10

۳ تألیفی — غلط یا حدسی؟ ☐

```
char s[] = "computer";
printf("%c%c%c", *s, *(s + 3), s[strlen(s) - 1]);
```

۱) cpr　۲) cmr　۳) cpe　۴) cur

۴ تألیفی — غلط یا حدسی؟ ☐

```
int m[2][3] = {{1,2,3},{4,5,6}};  int *p = &m[0][0];
printf("%d %d", *(p + 4), m[1][0] + *(*m + 2));
```

۱) 5 7　۲) 4 6　۳) 5 6　۴) 6 7

۵ تألیفی — غلط یا حدسی؟

```
void f(int *x, int y) { *x += y; y += *x; }
// main:
int a = 2, b = 3;  f(&a, b);  printf("%d %d", a, b);
```

۱) 5 8　۲) 5 3　۳) 2 3　۴) 2 8

۶ تألیفی — غلط یا حدسی؟

```
char *p = "abcde";  p += 2;
printf("%s %zu", p, strlen(p));
```

۱) cde 3　۲) cde 5　۳) c 1　۴) abcde 5

۷ تألیفی — غلط یا حدسی؟

```
int a[] = {10, 20, 30};  int *p = a;
printf("%d", *(p + 1) + *p + 1);
```

۱) 31　۲) 50　۳) 40　۴) 21

۸ تألیفی — غلط یا حدسی؟

(سیستم ۶۴ بیتی)

```
int a[10];  int *p = a;
printf("%zu %zu", sizeof(a) / sizeof(a[0]), sizeof(p));
```

۱) 10 40　۲) 10 8　۳) 40 8　۴) 10 10

۹ تألیفی — غلط یا حدسی؟

```
char s[10] = "ab";  strcat(s, "cd");
printf("%s %zu %zu", s, strlen(s), sizeof(s));
```

۱) abcd 4 5　۲) abcd 4 10　۳) abcd 5 10　۴) ab 2 10

۱۰ تألیفی

غلط یا حدسی؟

```
int x = 1, y = 2;  int *p = &x, *q = &y;
*p = *q + 1;  p = q;  *p = 7;
printf("%d %d", x, y);
```

۱) 7 2
۲) 3 7
۳) 3 2
۴) 7 7

کارت خلاصهٔ فصل ۵

برای مرور سریع، شب قبل از آزمون

- ✔ `*p++` اشاره‌گر را جلو می‌برد. `(*p)++` و `++*p` مقدار را زیاد می‌کنند.
- ✔ `a[i] = *(a+i)` . `p+1` یک **عضو** جلو می‌رود. `q - p` تعداد عضوهاست.
- ✔ دوبعدی سطری است: `*(*(b+i)+j) = b[i][j]` . `sizeof(b[0])` یک سطر است.
- ✔ `sizeof` آرایه کل بایت‌هاست، `sizeof` اشاره‌گر ۸ است. `strlen` بدون `'\0'` می‌شمارد. `char s[]` قابل تغییر است و `char *t = "…"` نه.

پاسخ‌نامهٔ فصل ۵

تا تمرین‌ها و آزمون فصل را خودت حل نکرده‌ای، این صفحه‌ها را نخوان.

تمرین فصل ۵

برگشت ← صفحهٔ ۵۸

۱. `11 20 40 30` (`*p + 1` یعنی ۱۰ + ۱، نه عضو بعدی)

۲. `7 10`

۳. `cdef e`

آزمون فصل ۵ — کلید و راه‌حل

برگشت ← صفحهٔ ۵۹

سؤال ۱	سؤال ۲	سؤال ۳	سؤال ۴	سؤال ۵	سؤال ۶	سؤال ۷	سؤال ۸	سؤال ۹	سؤال ۱۰
۱	۲	۱	۱	۲	۱	۱	۲	۲	۲

۱. p به `a[3] = 4` اشاره می‌کند و فاصله‌اش از ابتدا ۳ **عضو** است (نه ۱۲ بایت).

۲. `*p++ += 1` یعنی `*(p++) += 1`: `a[0]` می‌شود ۶ و p به `a[1]` می‌رود.

۳. `s[0] = 'c'`، `s[3] = 'p'`، و آخرین کاراکتر `'r'` است.

۴. `p + 4` در ترتیب سطری `m[1][1] = 5` است. `*(*m + 2)` همان `m[0][2] = 3` است، پس `4 + 3 = 7`.

۵. x اشاره‌گر است، پس a می‌شود ۵. y کپی است، پس b همان ۳ می‌ماند.

۶. از `'c'` تا آخر: `cde` با طول ۳.

۷. `*(p+1) = 20` و `*p + 1 = 11`. جمع ۳۱.

۸. ۱۰ عضو؛ اشاره‌گر ۸ بایت.

۹. strlen تعداد کاراکترها را تا `'\0'` می‌شمارد (۴)؛ sizeof اندازهٔ آرایه است (۱۰).

۱۰. x می‌شود `2 + 1 = 3`. بعد p به y اشاره می‌کند، پس `y = 7`.

فصل ۶

۶ struct، union، حافظهٔ پویا و لیست پیوندی

۴ درس | ۱۰ تست تألیفی در آزمون فصل

💡 **چرا این فصل؟** این فصل پل بین برنامه‌سازی و ساختمان داده است. تست‌ها معمولاً یکی از این‌ها را می‌پرسند: کپی شدن struct، اندازهٔ (padding) struct، `malloc` و `calloc` ، و ردگیری لیست پیوندی.

📐 **فرض‌ها** `int` ۴ بایتی، `double` و اشاره‌گر ۸ بایتی، تراز (alignment) پیش‌فرض gcc.

✅ **اعتبار** خروجی همهٔ کدها با `gcc` اجرا و چک شده است. (مثال‌های تکمیلی: `کد/مثال-ها.c`)

⏱️ **زمان مطالعه** یک بلوک ۹۰ دقیقه‌ای.

نقشهٔ فصل: ۱ struct | ۲ اندازهٔ struct و union | ۳ حافظهٔ پویا | ۴ لیست پیوندی در C | 📝 آزمون فصل

درس ۱ struct

از صفر: struct مثل یک فرم

آرایه چند جعبهٔ **هم‌نوع** است. اما گاهی چند اطلاعات **از نوع‌های مختلف** دربارهٔ یک چیز داریم؛ مثل کارت دانشجویی که شماره (عدد صحیح) و معدل (اعشاری) دارد. `struct` یک «فرم» با چند خانهٔ اسم‌دار (فیلد) می‌سازد.

```
struct Student { int id; double avg; };    // تعریف قالب فرم
struct Student s = {7, 16.5};              // یک فرم پر شده
printf("%d", s.id);                        // 7   («نقطه = «فیلدِ …»)
```

دسترسی با نقطه و فلش

- برای **متغیر** struct: `s.avg` .
- برای **اشاره‌گر** به struct: `p→avg` ، که کوتاه‌شدهٔ `(*p).avg` است: «برو به آدرس داخل p، بعد فیلد avg را بردار».

✏️ **مثال**

```
struct Student *p = &s;
p→avg = 18;
printf("%d %.1f", s.id, s.avg);    // 7 18.0
```

p به خود s اشاره می‌کند، پس تغییر از راه p، خود s را عوض کرد.

قاعده‌های تستی

```
struct P { int x, y; };
struct P a = {1, 2}, b;
b = a;            // کپی می‌شود کل struct
b.x = 9;          // a.x همان ۱ می‌ماند
```

- **انتساب struct کپی کامل است**، حتی اگر داخلش آرایه باشد. آرایهٔ معمولی را نمی‌شود با `=` کپی کرد، ولی آرایهٔ داخل struct همراه struct کپی می‌شود.
- **ارسال struct به تابع با مقدار است** (فتوکپی؛ فصل ۳). برای تغییرش اشاره‌گر بفرست: `void mv2(struct P *p) { p->x = 100; }`.
- آرایه‌ای از struct: `p++` به struct بعدی می‌رود و `(p+1)->x` فیلد عضو بعدی است.

✏️ مثال (آرایهٔ داخل struct کپی می‌شود)

```
struct S { int a[3]; };
struct S s1 = {{1, 2, 3}}, s2;
s2 = s1;          // مجاز است، با اینکه داخلش آرایه است
s2.a[0] = 9;
printf("%d %d", s1.a[0], s2.a[0]);   // 1 9
```

درس ۲ اندازهٔ struct و union

از صفر: چرا sizeof یک struct بیشتر از جمع فیلدهاست؟

پردازنده حافظه را تکه‌تکه می‌خواند و داده‌ای که از وسط دو تکه رد شود را کند می‌خواند. پس کامپایلر هر فیلد را در آدرسی می‌گذارد که **مضرب اندازهٔ خودش** باشد و بینشان بایت‌های خالی (padding) می‌گذارد. مثل پارکینگی که ماشین‌های بزرگ فقط در جای‌هایی با شمارهٔ مضرب ۴ یا ۸ پارک می‌کنند و بعضی جاها خالی می‌مانند.

✏️ مثال

در `struct Student { int id; double avg; }` : id در بایت ۰ تا ۳؛ avg باید از مضرب ۸ شروع شود ⇐ بایت ۴ تا ۷ خالی؛ avg در ۸ تا ۱۵ ⇐ `sizeof = 16` ، نه ۱۲.

قاعدهٔ padding

۱. هر فیلد باید در آدرسی مضرب **اندازهٔ خودش** شروع شود (char: ۱، short: ۲، int: ۴، double: ۸).

۲. اندازهٔ کل struct به مضرب **بزرگ‌ترین تراز** فیلدهایش گرد می‌شود (تا در آرایه‌ای از struct، عضو بعدی هم درست بنشیند).

struct	چینش	sizeof
`{char c; int i; char d;}`	`c` + ۳ پر کن + `i` (۴) + `d` + ۳ پر کن	**۱۲**
`{int i; char c; char d;}`	`i` (۴) + `c` + `d` + ۲ پر کن	**۸**
`{char c; double d;}`	`c` + ۷ پر کن + `d` (۸)	**۱۶**

💡 **نتیجه**

ترتیب فیلدها اندازه را عوض می‌کند. فیلدهای بزرگ را اول بگذار.

💡 **روش**

یک خط‌کش بایت بکش (۰، ۱، ۲، …). فیلدها را به ترتیب بچین؛ هر فیلد را به اولین خانه‌ای ببر که مضرب اندازه‌اش است. آخر کار، طول کل را به مضرب بزرگ‌ترین فیلد گرد کن.

✏️ **مثال (حساب کردن قدم‌به‌قدم)**

- `{char c; short s; int i;}` : c در ۰؛ s باید زوج باشد ⇐ ۱ بایت پر کن، s در ۲ و ۳؛ i در ۴ تا ۷ ⇐ **۸**.
- `{double d; char c;}` : d در ۰ تا ۷، c در ۸ ⇐ ۹ بایت که به مضرب ۸ گرد می‌شود ⇐ **۱۶**.

union: چند اسم برای یک حافظه

در union همهٔ فیلدها **از یک نقطه** شروع می‌شوند و روی هم می‌افتند؛ هر لحظه فقط یکی معنی دارد. اندازه = بزرگ‌ترین فیلد، گردشده به تراز. مثلاً `union {int i; char c[8];}` ۸ بایت است.

✏️ **مثال (حافظهٔ مشترک)**

```
union U { int i; char c[4]; } u;     // sizeof = 4
u.i = 0x41424344;                    // 44 43 42 41 :بایت‌ها (یعنی A B C D)
printf("%c", u.c[0]);                // D  روی x86 (little-endian: بایت کم‌ارزش اول)
```

نوشتن در `u.i` همان بایت‌هایی را عوض کرد که `u.c` می‌خواند.

درس ۳ حافظهٔ پویا

از صفر: اجاره کردن حافظه وقتی اندازه را از قبل نمی‌دانی

`int a[100];` اندازه‌اش موقع نوشتن برنامه ثابت است و آخر تابع هم پاک می‌شود. اگر اندازه را فقط موقع اجرا بدانی (مثلاً کاربر می‌گوید چند عدد دارد)، از **heap** حافظه **اجاره** می‌کنی:

- `malloc` حافظه را اجاره می‌کند و **آدرسش** را برمی‌گرداند (پس جوابش را در اشاره‌گر نگه می‌داری).

- این حافظه تا وقتی خودت `free` نکنی، می‌ماند؛ حتی بعد از تمام شدن تابع.
- `free` یعنی کلید را پس بدهی. بعد از آن دیگر حق نداری از آن استفاده کنی.

✏️ **مثال**

```
int n = 4;
int *a = malloc(n * sizeof(int));          // آرایهٔ ۴ تایی روی heap
for (int i = 0; i < n; i++) a[i] = i * i;  // 0 1 4 9
printf("%d %d", a[3], a[1]);               // 9 1
free(a);
```

بعد از a ،malloc را دقیقاً مثل آرایه با `a[i]` استفاده می‌کنی (فصل ۵).

تابع	کار
`malloc(n)`	n **بایت** می‌دهد؛ **مقدار اولیه ندارد** (زباله)
`calloc(k, size)`	k عضو می‌دهد، **همه صفر**
`realloc(p, n)`	اندازه را عوض می‌کند و مقادیر قبلی حفظ می‌شوند (ممکن است آدرس عوض شود)
`free(p)`	آزاد می‌کند

✏️ **مثال**

```
int *p = calloc(3, sizeof(int));     // 0 0 0
p[0] = 4;
p = realloc(p, 5 * sizeof(int));     // p[0] هنوز 4 است؛ p[3] و p[4] زباله‌اند
free(p);
```

سه خطای کلاسیک

- **نشت حافظه:** آدرس حافظه را گم کنی بدون اینکه free کنی (مثل اینکه کلید انبار را گم کنی؛ انبار تا آخر برنامه اجاره‌شده می‌ماند).
- **اشاره‌گر آویزان:** بعد از `free(p)` ، استفاده از p یا هر کپی‌اش (`q = p`) رفتار تعریف‌نشده است (کلید را پس داده‌ای ولی هنوز می‌روی در انبار).
- **free دوباره:** رفتار تعریف‌نشده.

✏️ مثال برای هر خطا

```
int *p = malloc(4);
p = malloc(8);          // نشت: آدرس ۴ بایت اول برای همیشه گم شد
int *q = p;
free(p);
*q = 1;                 // آویزان: q به حافظهٔ آزادشده اشاره می‌کند
free(q);                // دوباره: همان حافظه دوباره آزاد می‌شود
```

درس ۴ لیست پیوندی در C

از صفر: قطار با واگن‌های قلاب‌دار

آرایه خانه‌هایش پشت سر هم است. لیست پیوندی مثل **قطار** است: هر واگن (گره) یک **مقدار** و یک **قلاب** دارد که آدرس واگن بعدی است. واگن‌ها هر جای حافظه می‌توانند باشند؛ فقط قلاب‌ها ترتیب را نگه می‌دارند.

- اشاره‌گر `head` به واگن اول اشاره می‌کند.
- قلاب واگن آخر `NULL` است، یعنی «بعدی ندارد».

```
struct node { int v; struct node *next; };

struct node *push(struct node *h, int v) {          // درج در ابتدا
    struct node *n = malloc(sizeof *n);
    n->v = v;  n->next = h;  return n;
}
int count(struct node *h) { return h ? 1 + count(h->next) : 0; }

struct node *rev(struct node *h) {                  // وارون در O(n)
    struct node *prev = NULL;
    while (h) { struct node *nx = h->next; h->next = prev; prev = h; h = nx; }
    return prev;
}
```

✏️ مثال (ردگیری push)

شروع با `h = NULL` .

- `push(h, 1)` : واگن تازه با ۱ که قلابش به h قبلی (NULL) است ⇐ `1` .
- `push(h, 2)` : واگن ۲ جلوی ۱ می‌نشیند ⇐ `2 → 1` .
- `push(h, 3)` ⇐ `3 → 2 → 1` .

چاپ با `for (t = h; t; t = t->next) printf("%d ", t->v);` ⇐ `3 2 1` : ترتیب **برعکس** ورود، چون هر بار در ابتدا اضافه کردیم. بعد از `rev` می‌شود `1 → 2 → 3` .

✏️ مثال (ردگیری rev روی ۱ → ۲)

با `prev = NULL` و `h = 1` : قلاب ۱ به NULL برمی‌گردد، h = ۲، prev = ۱؛ قلاب ۲ به ۱ برمی‌گردد، prev = ۲، h = NULL. حاصل `1 → 2` .

✍️ تمرین فصل ۶

اول خودت حل کن، بعد پاسخ‌نامه

۱. `sizeof(struct {char a; char b; int c;})` چند است؟

۲. با `struct P arr[3] = {{1,2},{3,4},{5,6}}; struct P *p = arr; p++;` ، مقدار `p→y` و `(p+1)→x` چیست؟

🔑 پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: **صفحهٔ ۷۲**

📝 آزمون فصل ۶

تست‌های تألیفی (خروجی همه با gcc اجرا و تأیید شده)

۱۰ سؤال | ⏱️ زمان: ۱۲ دقیقه | 🔑 کلید و راه‌حل: صفحهٔ ۷۲

تست‌های **تألیفی**. خروجی همه با `gcc` تأیید شده است.

✔️ مثل جلسهٔ کنکور: زمان بگیر، روی هیچ سؤالی بیشتر از ۴ دقیقه نمان، و در مربع کنار سؤال‌های غلط یا حدسی علامت بزن تا در دور دوم فقط همان‌ها را حل کنی.

۱ تألیفی — غلط یا حدسی؟ ☐

```c
struct P { int x, y; };
struct P a = {1, 2}, b;  b = a;  b.x = 9;
printf("%d %d", a.x, b.x);
```

۱) `9 9`
۲) `1 9`
۳) `1 1`
۴) خطای کامپایل

۲ تألیفی — غلط یا حدسی؟ ☐

```c
void mv(struct P p)   { p.x = 100; }
void mv2(struct P *p) { p->x = 100; }
// main:
struct P a = {1, 2};  mv(a);  printf("%d ", a.x);  mv2(&a);  printf("%d", a.x);
```

۱) `100 100`
۲) `1 1`
۳) `1 100`
۴) `100 1`

۳ تألیفی — غلط یا حدسی؟ ☐

`sizeof(struct S { char c; int i; char d; })` ؟

۱) ۶
۲) ۸
۳) ۱۲
۴) ۱۶

۴ تألیفی — غلط یا حدسی؟ ☐

`sizeof(struct T { int i; char c; char d; })` ؟

۱) ۶
۲) ۸
۳) ۱۲
۴) ۱۶

۵ تألیفی — غلط یا حدسی؟

`sizeof(union U { int i; char c[8]; })`؟

۱) ۴ ۲) ۸ ۳) ۱۲ ۴) ۱۶

۶ تألیفی — غلط یا حدسی؟

```
int *p = calloc(4, sizeof(int));  p[1] = 5;
printf("%d", p[0] + p[1] + p[3]);
```

۱) ۵ ۲) مقدار نامعلوم ۳) ۰ ۴) ۱۵

۷ تألیفی — غلط یا حدسی؟

بعد از `h = push(push(push(NULL, 1), 2), 3)` (تابع push درس ۴)، پیمایش لیست از h چه چاپ می‌کند؟

۱) `1 2 3` ۲) `3 2 1` ۳) `3` ۴) `1`

۸ تألیفی — غلط یا حدسی؟

```
struct A { int v[3]; };
struct A a = {{1, 2, 3}}, b = a;  b.v[0] = 9;
printf("%d %d", a.v[0], b.v[0]);
```

۱) `9 9` ۲) `1 9`
۳) خطای کامپایل (آرایه کپی نمی‌شود) ۴) `1 1`

۹ تألیفی — غلط یا حدسی؟

```
struct P arr[3] = {{1,2},{3,4},{5,6}};  struct P *p = arr;  p++;
printf("%d %d", p->y, (p + 1)->x);
```

۱) `2 3` ۲) `4 5` ۳) `3 4` ۴) `4 6`

۱۰ تألیفی

غلط یا حدسی؟

```
int *p = malloc(3 * sizeof(int));
for (int i = 0; i < 3; i++) p[i] = i * i;
p = realloc(p, 5 * sizeof(int));  p[4] = 7;
printf("%d %d", p[2], p[4]);
```

۱) `4 7`
۲) `0 7`
۳) مقدار نامعلوم برای `p[2]`
۴) `2 7`

🗂️ کارت خلاصهٔ فصل ۶

برای مرور سریع، شب قبل از آزمون

- ✔ انتساب و ارسال struct کپی است (حتی آرایهٔ داخلش). برای تغییر، اشاره‌گر بفرست و `->` بزن.
- ✔ padding: هر فیلد روی مضرب اندازهٔ خودش؛ کل struct مضرب بزرگ‌ترین تراز. `{char,int,char} = 12` و `{int,char,char} = 8`.
- ✔ union = بزرگ‌ترین فیلد. malloc زباله می‌دهد و calloc صفر. realloc مقادیر را حفظ می‌کند. بعد از free استفاده نکن.
- ✔ درج در ابتدای لیست ترتیب را برعکس می‌کند. وارون کردن با سه اشاره‌گر `prev`، `h` و `nx` در `O(n)`.

پاسخ‌نامهٔ فصل ۶

تا تمرین‌ها و آزمون فصل را خودت حل نکرده‌ای، این صفحه‌ها را نخوان.

تمرین فصل ۶

برگشت ← صفحهٔ ۶۸

۱. `8` (`a`، `b`، ۲ بایت پر کن، `c`)

۲. `4 5`

آزمون فصل ۶ — کلید و راه‌حل

برگشت ← صفحهٔ ۶۹

سؤال ۱	سؤال ۲	سؤال ۳	سؤال ۴	سؤال ۵	سؤال ۶	سؤال ۷	سؤال ۸	سؤال ۹	سؤال ۱۰
۲	۳	۳	۲	۲	۱	۲	۲	۲	۱

۱. انتساب struct کپی است و a دست نمی‌خورد.

۲. mv روی کپی کار می‌کند و mv۲ از طریق اشاره‌گر، خود a را عوض می‌کند.

۳. `1 + 3 (پر کن) + 3 + 1 + 4 (پر کن) = 12`

۴. `2 + 1 + 1 + 4 (پر کن) = 8`

۵. بزرگ‌ترین فیلد ۸ بایت است.

۶. calloc صفر می‌کند: `0 + 5 + 0 = 5`.

۷. درج در ابتدا ترتیب را برعکس می‌کند: `1 2 3`.

۸. آرایهٔ داخل struct همراه struct کپی می‌شود.

۹. p به `arr[1] = {3,4}` اشاره می‌کند، پس `p→y = 4` و `(p+1)→x = 5`.

۱۰. realloc مقادیر قبلی را حفظ می‌کند: `p[2] = 4` و `p[4] = 7`.